computers

MDPI

Systematic Review

The Influence of Artificial Intelligence Tools on Learning Outcomes in Computer Programming: A Systematic Review and Meta-Analysis

Manal Alanazi [1], Ben Soh [1,*], Halima Samra [2] and Alice Li [3]

[1] Department of Computer Science and Information Technology, La Trobe University, Melbourne 3086, Australia; manalmohammedg.alanazi@latrobe.edu.au
[2] Computer Science & Information Technology, King Abdulaziz University, Jeddah 21589, Saudi Arabia; hsamra@kau.edu.sa
[3] La Trobe Business School, La Trobe University, Melbourne 3086, Australia; a.li@latrobe.edu.au
* Correspondence: b.soh@latrobe.edu.au

Abstract: This systematic review and meta-analysis investigates the impact of artificial intelligence (AI) tools, including ChatGPT 3.5 and GitHub Copilot, on learning outcomes in computer programming courses. A total of 35 controlled studies published between 2020 and 2024 were analysed to assess the effectiveness of AI-assisted learning. The results indicate that students using AI tools outperformed those without such aids. The meta-analysis findings revealed that AI-assisted learning significantly reduced task completion time (SMD = −0.69, 95% CI [−2.13, −0.74], I^2 = 95%, p = 0.34) and improved student performance scores (SMD = 0.86, 95% CI [0.36, 1.37], p = 0.0008, I^2 = 54%). However, AI tools did not provide a statistically significant advantage in learning success or ease of understanding (SMD = 0.16, 95% CI [−0.23, 0.55], p = 0.41, I^2 = 55%), with sensitivity analysis suggesting result variability. Student perceptions of AI tools were overwhelmingly positive, with a pooled estimate of 1.0 (95% CI [0.92, 1.00], I^2 = 0%). While AI tools enhance computer programming proficiency and efficiency, their effectiveness depends on factors such as tool functionality and course design. To maximise benefits and mitigate over-reliance, tailored pedagogical strategies are essential. This study underscores the transformative role of AI in computer programming education and provides evidence-based insights for optimising AI-assisted learning.

Keywords: artificial intelligence (AI); ChatGPT; computer programming education; learning outcomes; GitHub Copilot; task efficiency; student engagement; meta-analysis

check for updates

Academic Editor: Pedro Alonso Jordá

Received: 13 April 2025
Revised: 1 May 2025
Accepted: 6 May 2025
Published: 9 May 2025

Citation: Alanazi, M.; Soh, B.; Samra, H.; Li, A. The Influence of Artificial Intelligence Tools on Learning Outcomes in Computer Programming: A Systematic Review and Meta-Analysis. *Computers* **2025**, *14*, 185. https://doi.org/10.3390/computers14050185

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Artificial intelligence (AI) has emerged as a transformative force in education, offering innovative approaches to enhance learning through personalisation and automation. Among the various AI technologies, conversational agents, such as ChatGPT, have garnered significant attention for their potential to revolutionise teaching and learning [1]. The complexity of computer programming—characterised by intricate syntax, logical reasoning, and problem-solving—often presents significant challenges for students, particularly in introductory courses. Many students struggle to bridge the gap between theoretical programming concepts and their practical implementation in code [2]. AI-powered tools have the potential to address these challenges by providing real-time feedback, guiding students through the nuances of error correction and fostering a more interactive and

personalised learning experience. As a result, AI-driven platforms, such as ChatGPT, are increasingly being integrated into computer programming curricula to offer customised support and real-time guidance, supplementing traditional teaching methods that may struggle to provide individualised attention [3].

The role of AI in computer programming education is multifaceted. On the one hand, AI tools, including intelligent tutoring systems (ITSs), have long been utilised to create personalised learning experiences, with research consistently demonstrating their effectiveness in improving student learning outcomes, particularly in STEM disciplines such as mathematics and computer science [4]. These systems dynamically adapt to individual learning paces and provide tailored feedback, thereby assisting students in overcoming specific conceptual and technical challenges. More recently, AI models, such as ChatGPT, which generate human-like conversational responses based on extensive datasets, have been adopted in educational settings for problem-solving, code debugging, and conceptual learning [5]. Unlike ITSs, ChatGPT and similar chatbot-based AI tools do not rely on a fixed set of pre-programmed responses, making them highly flexible and capable of addressing a broad range of student inquiries. Preliminary research on the application of GPT-3 in tutoring programming suggests that AI-driven resources can enhance students' problem-solving abilities, increase engagement, and deepen their understanding of computer programming concepts [6,7]. However, despite these promising results, concerns have been raised regarding the potential over-reliance on AI-generated solutions, which could lead to a diminished focus on fundamental programming logic and critical thinking skills [8].

Despite the growing body of research supporting the role of AI in education, there remains a lack of comprehensive and structured reviews synthesising evidence from controlled experimental studies [9]. AI research has often been criticised for limitations such as small sample sizes, inconsistent experimental controls, and variations in AI tool functionalities, all of which challenge the generalisability of findings [10]. In addition, AI-assisted programming education encompasses a wide range of tools, from simple code auto-completion assistants to advanced machine learning-driven tutoring systems, making it difficult to draw broad conclusions from isolated studies. These gaps in the literature underscore the need for a systematic review and meta-analysis to aggregate evidence from diverse controlled experimental studies and provide a clearer understanding of how AI tools, particularly ChatGPT, impact learning outcomes in introductory programming courses [11].

This systematic review and meta-analysis aims to assess the impact of AI tools, such as ChatGPT, on student learning outcomes in introductory computer programming courses. The primary objectives of this study are to evaluate the effectiveness of AI in improving student performance, engagement, and problem-solving skills, as well as to compare AI-assisted learning with traditional instructional methods. Furthermore, this review examines how different AI tool designs, usage patterns, and student interactions influence learning outcomes. By synthesising findings from controlled experimental studies, this review seeks to provide evidence-based recommendations for effectively integrating AI tools into computer programming curricula to maximise their educational benefits while addressing potential challenges.

1.1. Research Aims

This study seeks to offer empirical insights to support educators and policymakers in the effective integration of artificial intelligence (AI) tools into computer programming curricula. The motivation stems from the growing presence of AI-driven tools, such as

ChatGPT and GitHub Copilot, in educational settings. Several key considerations underpin this investigation:

- The widespread availability of AI-powered coding assistants is reshaping how students approach programming tasks. It is essential to evaluate their tangible effects on learning outcomes.
- While these tools can enhance learning by improving efficiency, lowering cognitive load, and providing immediate support, they also pose potential drawbacks, such as fostering over-reliance, diminishing problem-solving skills, and promoting superficial understanding. A nuanced understanding of these trade-offs is vital.
- Despite their growing adoption, there is a noticeable lack of rigorous, data-driven studies assessing the pedagogical impact of AI tools in computer programming education, particularly in controlled experimental contexts.

1.2. Research Objectives

This paper makes the following key contributions:

- It synthesises evidence from 35 controlled studies conducted between 2020 and 2024 to provide a robust evaluation of AI-assisted learning in introductory computer programming courses.
- This study measures the impact of AI tools, such as ChatGPT and GitHub Copilot, on student performance, task completion time, and perceived ease of understanding, offering statistical insights into their educational effectiveness.
- While AI tools demonstrably improve task efficiency and programming performance, the findings reveal a limited effect on students' conceptual understanding and overall learning success, underscoring the importance of balanced integration.
- Aggregated feedback indicates high student acceptance and perceived usefulness of AI tools, highlighting their potential value in educational settings.
- This study provides actionable guidance for educators, recommending adaptive teaching strategies that leverage AI benefits while mitigating risks related to over-reliance.
- By identifying gaps in the current literature and limitations in existing AI-assisted learning approaches, this study establishes a foundation for future research into AI-driven educational practices.

2. Methods

This review was performed in accordance with the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) guidelines. It adhered to the methodologies proposed by Kitchenham [12] and illustrated in the Supplementary Materials.

2.1. Eligibility Criteria

This systematic review and meta-analysis was conducted to assess the impact of AI tools, such as ChatGPT, on learning outcomes in introductory computer programming courses. Studies were included based on specific eligibility criteria, requiring them to be peer-reviewed, present numerical data, and follow a well-defined study design. Only controlled experiments evaluating the effectiveness of AI tools in computer programming education were considered. The study selection process adhered to the PRISMA 2020 guidelines, ensuring transparency by systematically documenting the number of records retrieved, screened, included, and excluded at each stage.

2.2. Search Strategy

A comprehensive and systematic search was conducted across multiple academic databases, including Web of Science, Scopus, IEEE Xplore, ACM Digital Library, and

Lens.org. The search strategy incorporated a combination of keywords and Boolean operators (AND, OR) to refine the results. The primary search terms included "Artificial Intelligence", "AI tools", "ChatGPT", "learning outcomes", "introductory programming", "controlled experiments", and "educational technology". The search was restricted to peer-reviewed journal articles and conference papers published in English between 2000 and 2024. Studies were included if they provided quantitative data relevant to learning outcomes in introductory programming courses. Qualitative studies were filtered based on their relevance to AI-assisted learning in computer programming education. The detailed search strings and their corresponding databases are outlined in Table 1. To maintain methodological rigour, the search syntax was adapted slightly across databases to account for differences in indexing mechanisms, search field constraints, and Boolean logic handling. For example, IEEE Xplore requires simplification of nested queries due to its functional limitations, while Scopus allows for more complex semantic expansions. Despite these necessary adjustments, the core search structure and keyword concepts remained consistent across all databases, ensuring transparency, comparability, and reproducibility of the search strategy.

Table 1. Search strategy.

Database	Search String
Web of Science	("artificial intelligence" OR "AI" OR "ChatGPT") AND ("learning outcomes" OR "programming skills" OR "coding") AND ("introductory programming" OR "programming education") AND ("controlled experiment" OR "randomised controlled trial" OR "quasi-experimental")
Scopus	("artificial intelligence" OR "AI" OR "ChatGPT") AND ("learning outcomes") AND ("introductory programming") AND ("experimental study")
ACM	("artificial intelligence" OR "AI" OR "ChatGPT") AND ("learning outcomes") AND ("introductory programming") AND ("experimental study")
IEEE	("artificial intelligence" OR "AI" OR "ChatGPT") AND ("learning outcomes") AND ("introductory programming") AND ("experimental study") AND ("controlled experiment" OR "randomised controlled trial" OR "quasi-experimental")

2.3. Study Selection and Screening

The study selection process followed a two-stage screening approach. First, titles and abstracts were screened for relevance, followed by a full-text review to ensure compliance with inclusion criteria. Studies were included if they met the following conditions: (1) controlled experimental design evaluating AI tools such as ChatGPT in computer programming education, (2) provision of quantitative learning outcome metrics (e.g., test scores, task completion rates, code quality), and (3) publication in peer-reviewed journals or conferences. Studies were excluded if they lacked empirical data, were non-peer-reviewed, or did not specifically examine AI-assisted learning in computer programming. Only studies published in English were considered. The final dataset consisted of studies that met all inclusion criteria.

2.4. Inclusion and Exclusion Criteria

The inclusion criteria encompassed controlled experimental studies investigating the impact of AI tools on learning outcomes in introductory programming courses. Eligible studies had to report quantitative performance metrics, such as examination scores, code

accuracy, and computer programming task completion rates. Only studies published in English since 2010 were considered to ensure methodological relevance. Studies were excluded if they lacked a controlled experimental design, did not integrate AI tools into computer programming education, or did not report quantifiable learning outcomes. Additionally, non-peer-reviewed articles, conference abstracts, the grey literature, and studies published in languages other than English were excluded from this review.

2.5. Data Extraction

Data extraction contained the following: authors conducting the study, study design, for example, randomised controlled trial; geographical location of the research; characteristics of the population, for example, age, educational background, and experience level; details of the intervention, for example, specific AI tools used, such as ChatGPT; information about the control group, for example, the nature of the control or comparison group; and, finally, the outcomes evaluated, for example, test scores, performance metrics, and learning improvements. Each of these studies was carefully documented to allow for comparison and synthesis in the following meta-analysis.

2.6. Quality Assessment of Included Studies

The quality of the included studies was assessed using established tools. The Newcastle–Ottawa Scale (NOS) was used to assess the risk of bias in the non-randomised controlled trials, focusing on selection, comparability, and outcome assessment. The JBI Checklist was used to appraise the methodological quality of the experimental studies, ensuring rigorous study design and reporting. A questionnaire was also designed to assess whether the measured outcomes in the studies were appropriate, valid, reliable, and relevant to the learning outcomes in introductory programming. This meant that only high-quality studies contributed to this systematic review and meta-analysis. More details of the analysis are provided in Section 3.

2.7. Risk of Biased Assessment

The risk of bias was evaluated using the RoB 2 tool in Review Manager by focusing on selection bias, performance bias, detection bias, reporting bias, and other potential biases. For each domain, signalling questions were answered, and a level of risk of bias was assigned as follows: low, some concerns, or high. More details of the analysis are provided in Section 3.

2.8. Statistical Analysis

Meta-analytic techniques were applied using Review Manager 5, Stata 17, and SPSS v27. Effect sizes were computed for each study, with standardised mean differences (SMDs) and mean differences (MDs) used to quantify the impact of AI tools on learning outcomes in computer programming. A random-effects model was employed to account for variability among the studies. Heterogeneity was assessed using the I^2 statistic, with values exceeding 50% indicating substantial heterogeneity. Subgroup analyses were conducted to explore potential moderators, including study design, AI tool type, and participant characteristics.

To assess publication bias, a funnel plot and Egger's test were utilised. A sensitivity analysis was performed by systematically removing high-risk studies to evaluate the robustness of the findings. All effect sizes were reported with 95% confidence intervals (CIs), and statistical significance was set at $p < 0.05$ to ensure replicability and transparency in reporting. More details of the analysis are provided in Section 3.

3. Results

3.1. Study Characteristics and Geographic Distribution

A total of 32 studies were included in this systematic review and meta-analysis, following the rigorous selection criteria, as illustrated in the PRISMA flow diagram (Figure 1). The selected studies employed diverse experimental methodologies, with randomised controlled trials (RCTs) and quasi-experimental designs being the most commonly used approaches. These studies examined the role of AI-powered tools, including ChatGPT, GitHub Copilot, and other automated coding assistants, in enhancing real-time feedback, debugging support, and code generation in computer programming education. The geographic distribution of studies was broad, encompassing research conducted in the United States (14 studies), Europe (10 studies), and Asia (5 studies). The majority of the studies targeted undergraduate students, although some also included high school students and adult learners participating in online computer programming courses.

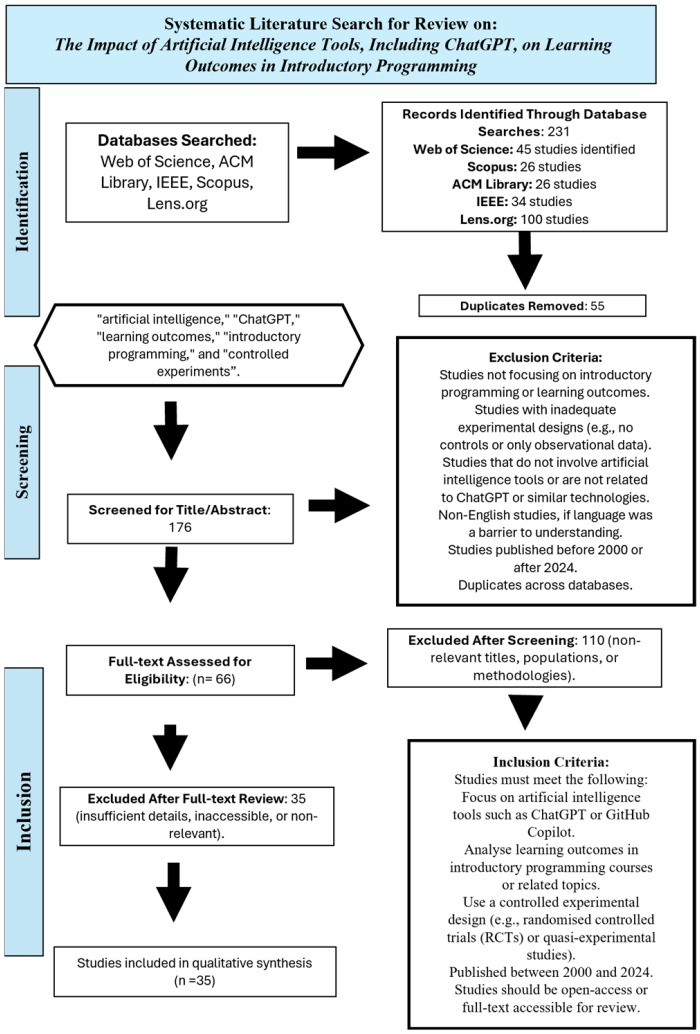

Figure 1. PRISMA flow diagram.

3.2. Bibliometric Analysis of Included Studies

To assess the research landscape and trends in AI-assisted computer programming education, a bibliometric analysis was conducted, visualising key authors, co-authorship networks, and term evolution in the field (Figure 2). These analyses highlighted the most influential researchers and thematic clusters, illustrating how research on AI in computer programming education has evolved over time.

Furthermore, an analysis of scholarly literature trends (Figure 3) was performed to understand the publication volume, citation distribution, and open-access availability of the relevant studies. The data indicate a significant increase in AI-related computer programming education research from 2020 onward, reflecting the growing interest in integrating AI tools into educational settings.

The citation and publication impact of AI tools in computer programming education was further analysed using Litmaps 2023 visualisation (Figure 4). This analysis demonstrates the connections between foundational studies and recent influential works, providing insight into the progression of research in this domain. The citation trends suggest that AI-driven educational tools are gaining substantial recognition in academic discourse.

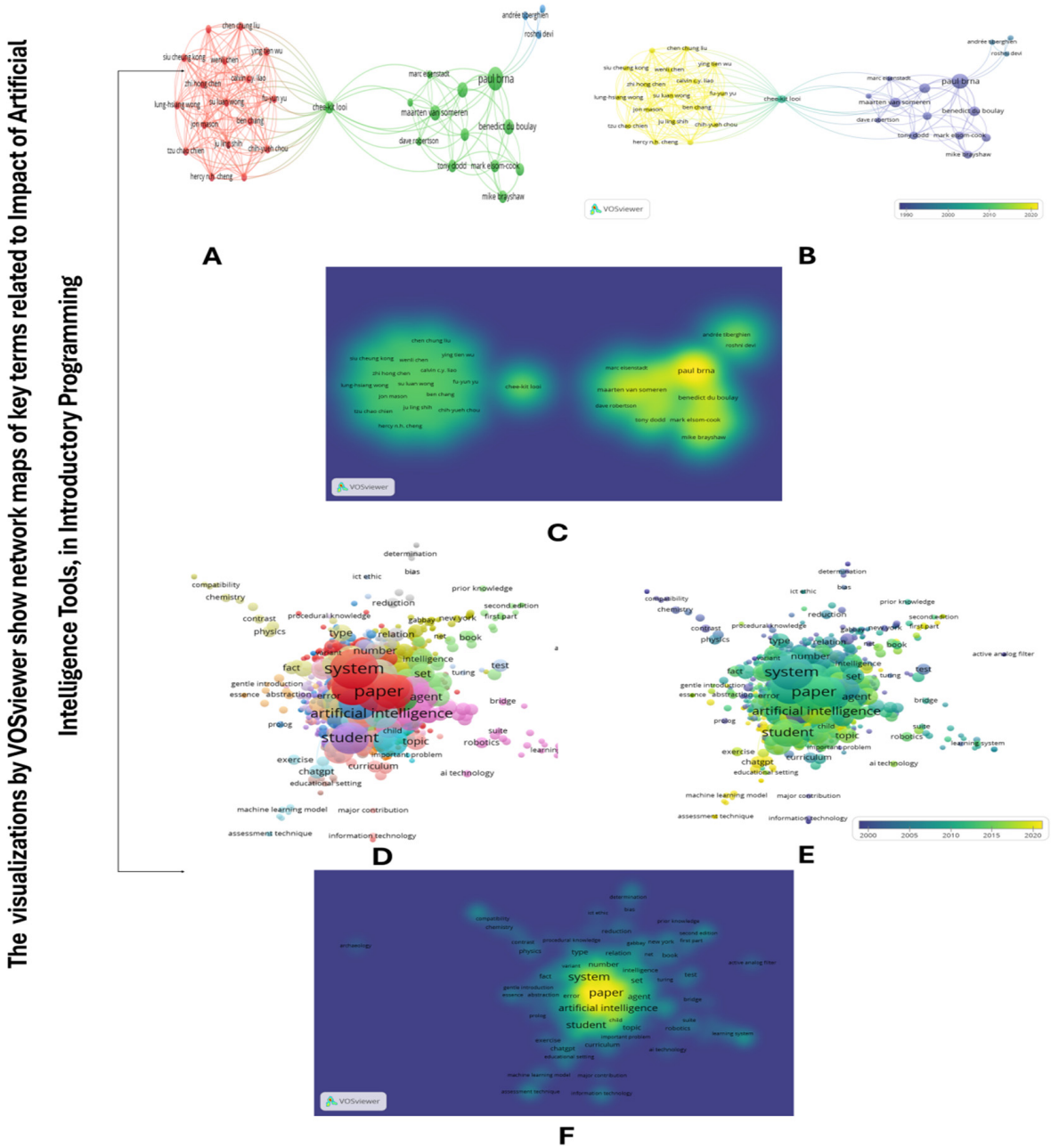

Figure 2. (**A**) Bibliographic analysis of authors. (**B**) Co-authorship and citation network. (**C**) Visualisations of key authors in the impact of artificial intelligence tools, including ChatGPT, on learning outcomes in introductory programming. (**D**) Term cluster MESH terms. (**E**) Chronological term evolution. (**F**) Term density distribution within the scientific literature.

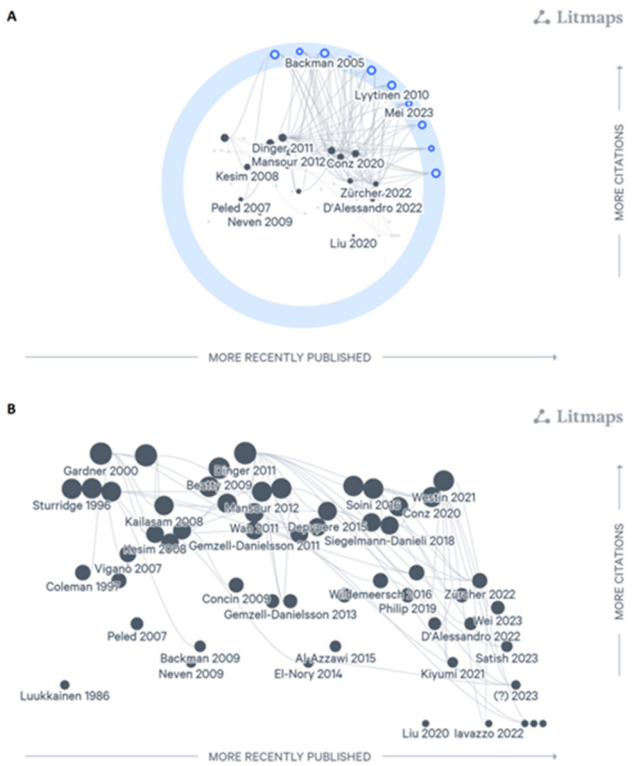

Figure 3. Citation and publication trends of AI tools in programming education.

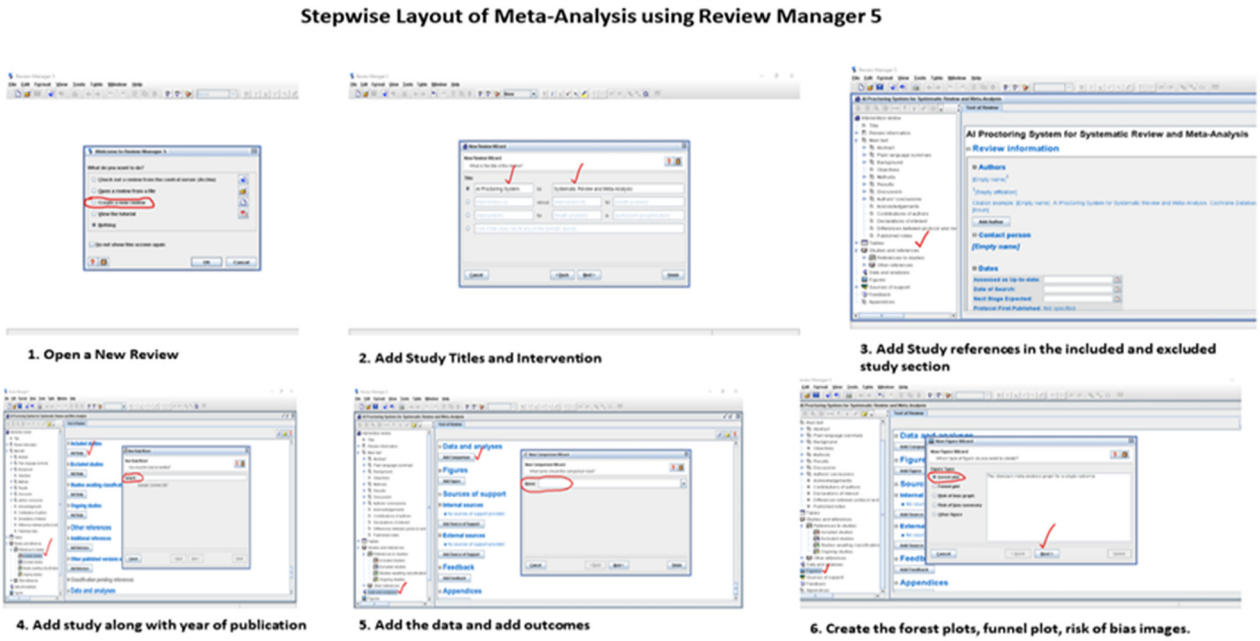

Figure 4. Stepwise layout of the meta-analysis using Review Manager 5.

3.3. Citation Analysis of Included Studies

To synthesise findings across studies, statistical analyses were performed using Review Manager 5, Stata 17, and SPSS v27 to evaluate the impact of AI-assisted learning on student performance, engagement, and efficiency. Figure 5 illustrates the stepwise process of conducting the meta-analysis, detailing the use of Review Manager 5 for data entry, study classification, and statistical computations. In addition, Figure 6 presents a schematic diagram outlining the data extraction and meta-analysis workflow, providing a conceptual overview of the methodological framework used to compare AI-enhanced learning with

traditional educational methods. A comprehensive summary of the 35 included studies, including their methodologies, intervention types, and key findings, is available in Table A1.

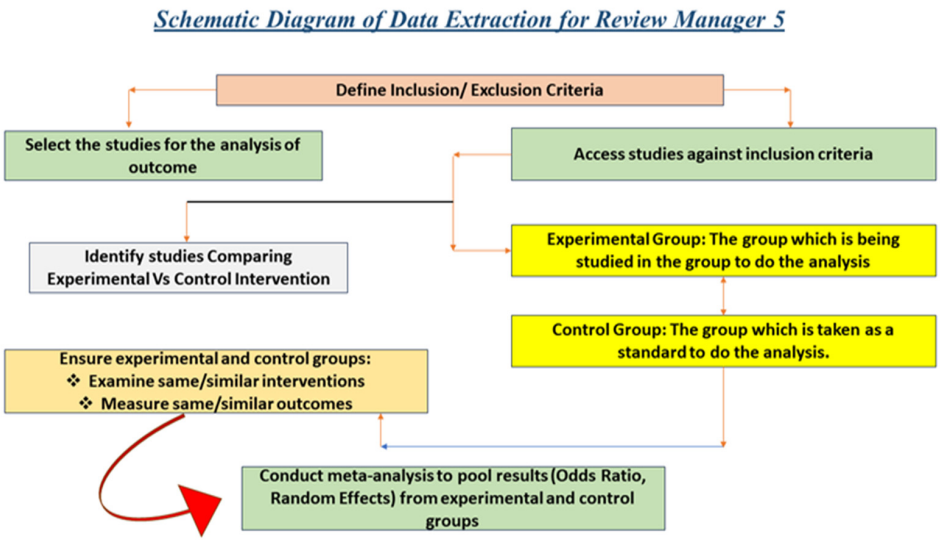

Figure 5. Schematic diagram of data extraction and meta-analysis tables for Review Manager 5.

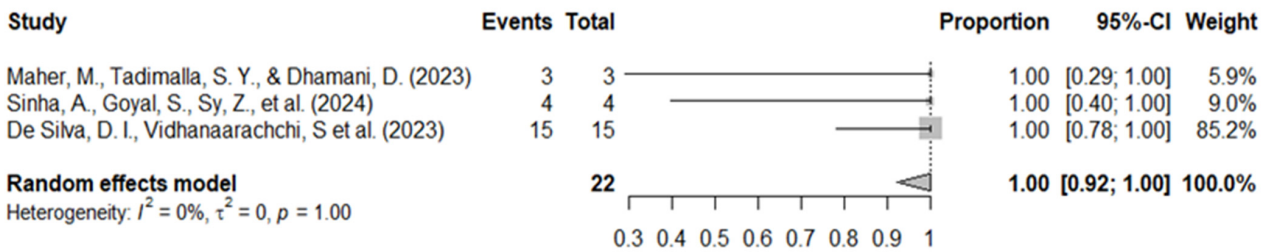

Figure 6. Forest plot [13–15].

3.4. Perceived Usefulness and Benefits of AI Tools

The outcome was based on data pooled from three studies [13–15]. The analysis assessing students' perceptions of AI tools in introductory programming courses showed a strong consensus on their usefulness and benefits. The overall pooled prevalence was 1.0 (95% CI [0.92–1.00]) [Figures 6 and 7], indicating that nearly all students found AI tools beneficial for their learning experience. The heterogeneity was low (I^2 = 0%), suggesting consistent findings across the included studies. These results highlight the widespread acceptance of AI tools among students, emphasising their perceived value in enhancing computer programming comprehension, problem-solving skills, and engagement.

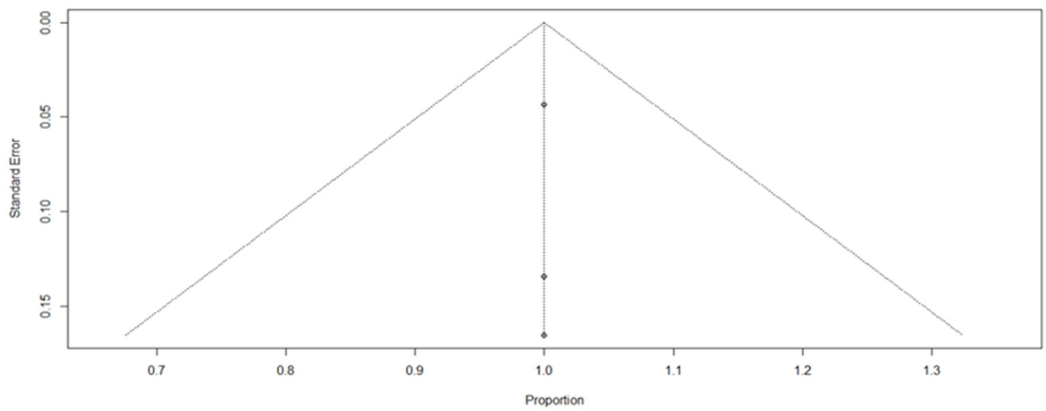

Figure 7. Funnel plot-perceived usefulness and benefits of AI tools.

3.5. Task Completion Time

The meta-analysis of four studies [16–19] comparing task completion time between students using AI-enabled learning tools and those in control groups demonstrated a moderate reduction in completion time, favouring AI-assisted learning. The pooled SMD was −0.69 (95% CI [−2.13, −0.74]) [Figures 8 and 9], indicating that students utilising AI tools completed computer programming tasks more quickly on average. However, substantial heterogeneity was observed (I^2 = 95%), suggesting considerable variability across studies in the effect of AI tools on task completion speed. The *p*-value for the overall effect was 0.34, which indicated that the reduction in completion time did not reach statistical significance. The high heterogeneity may be attributed to differences in AI tool design, student experience levels, and task complexity across the included studies. Despite this, the negative SMD suggested a general trend where AI-assisted learning may help students complete computer programming tasks more efficiently.

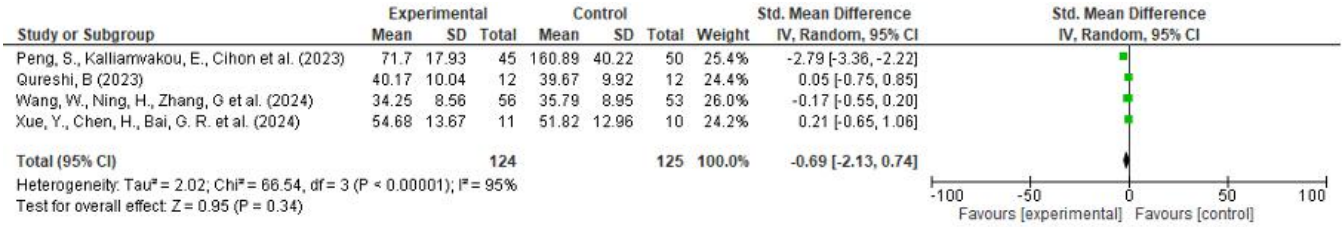

Figure 8. Forest plot [16–19].

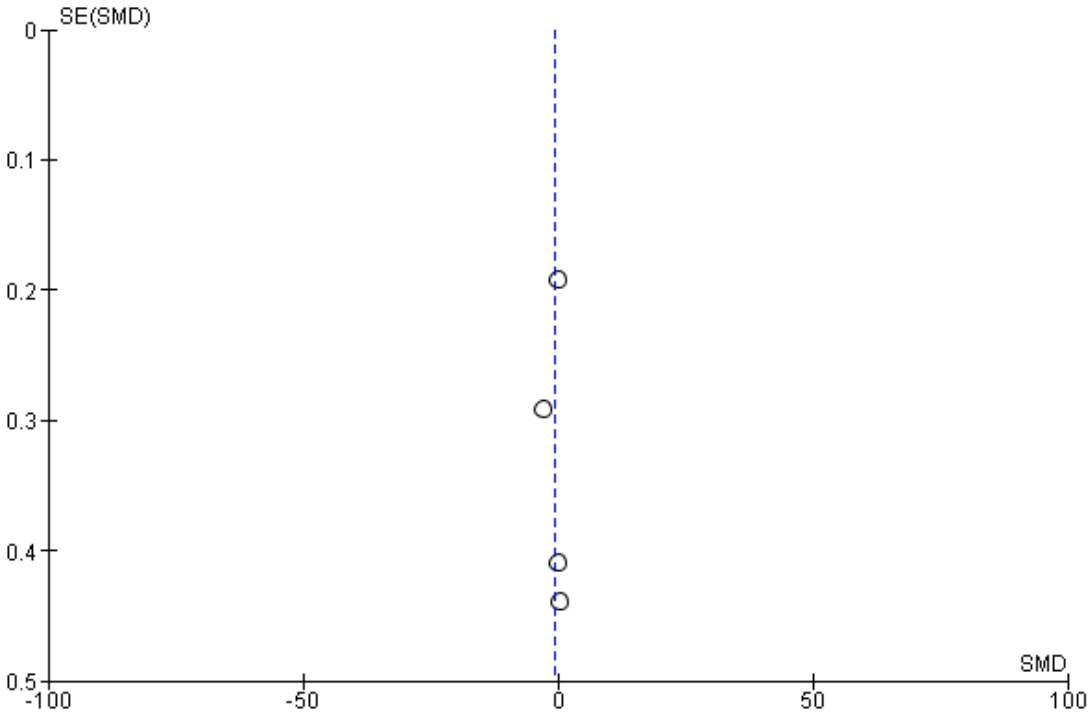

Figure 9. Forest plot-task completion time.

3.6. Success and Ease of Understanding

This analysis included three studies [20–22]. Comparing students' perceived success and ease of understanding in AI-enabled learning environments versus traditional methods yielded a small, non-significant effect in favour of AI tools. The pooled SMD was 0.16 (95% CI [−0.23, 0.55], *p* = 0.41) [Figures 10 and 11]. This suggested a slight improvement in perceived success and comprehension among students using AI tools. Moderate hetero-

geneity was observed (I^2 = 55%). This variability may stem from differences in how AI tools were implemented, the complexity of computer programming tasks, or students' prior experience with AI-assisted learning. While AI-enabled tools might enhance conceptual understanding for some learners, the overall effect remains uncertain based on the available data. Further research is needed to explore factors influencing students' perception of success and ease of understanding, such as the nature of AI-generated explanations, the degree of interaction with AI tools, and students' computer programming backgrounds.

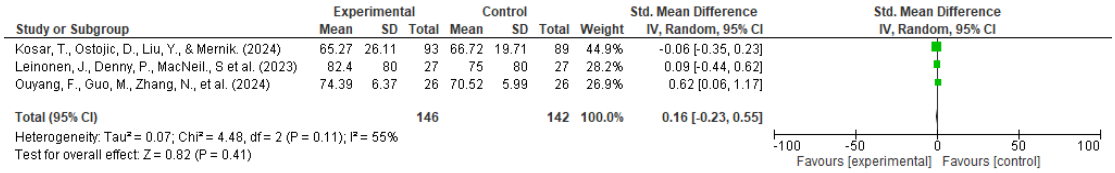

Figure 10. Forest plot [20–22].

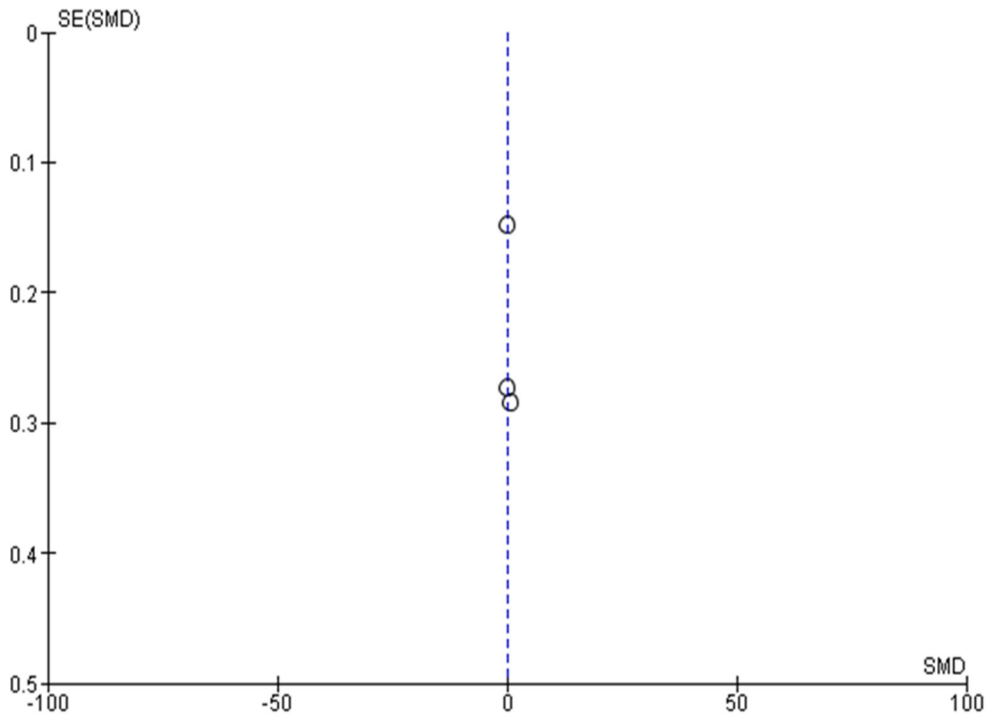

Figure 11. Funnel plot–success and ease of understanding.

3.7. Student Performance

The student performance analysis included data from four studies [17,19,23,24]. The impact of AI-enabled learning tools on student performance in introductory programming courses showed a significant positive effect. The pooled SMD was 0.86 (95% CI [0.36, 1.37], p = 0.0008) [Figures 12 and 13]. This indicated that students using AI-assisted learning tools scored notably higher compared to those in traditional learning environments. Moderate heterogeneity was observed (I^2 = 54%). This suggested some variation across studies, likely due to differences in AI tool design, level of AI assistance, or assessment methods used in each study. Despite this, the positive effect size (SMD = 0.86) suggested that AI tools contribute meaningfully to improving students' computer programming proficiency.

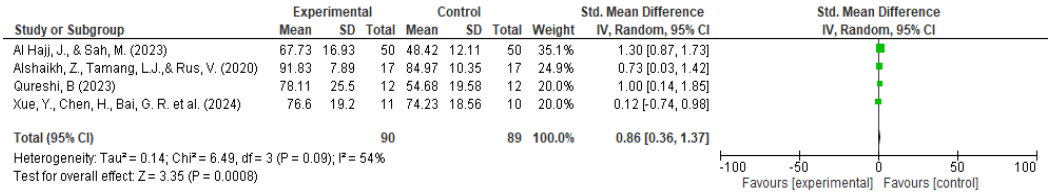

Figure 12. Forest plot [17,19,23,24].

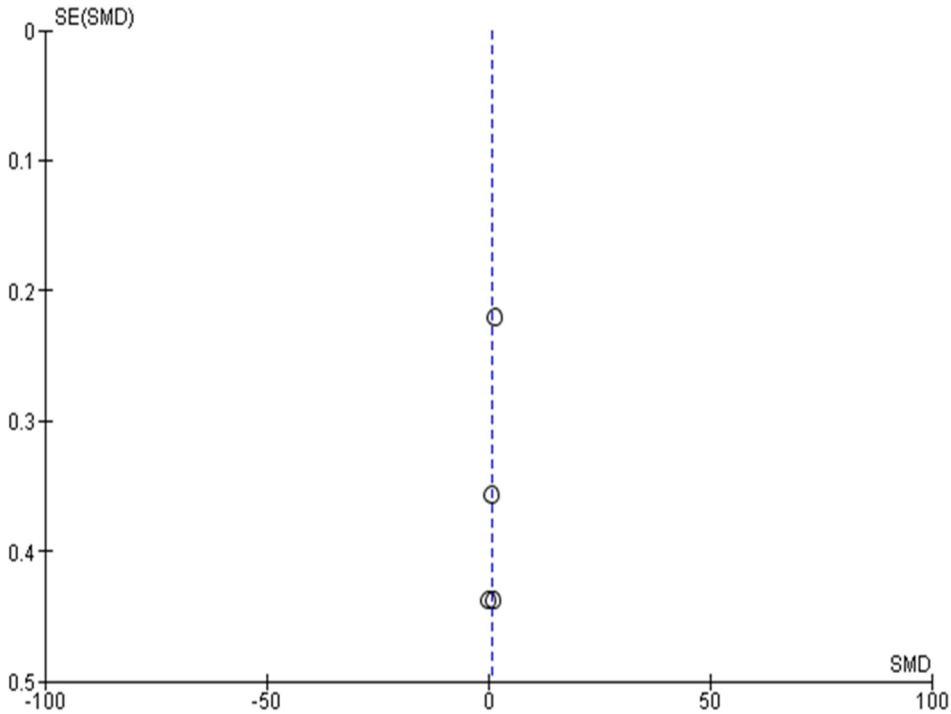

Figure 13. Funnel plot–student performance.

3.8. Sensitivity Analysis

3.8.1. Sensitivity Analysis for Task Completion Time

The sensitivity analysis for task completion time revealed significant variations among the included studies, confirming the robustness of the findings. The results remained consistent when individual studies were excluded, indicating that no single study had an undue influence on the overall conclusions. This provides reassurance regarding the reliability of the observed effect, suggesting that AI-enabled learning tools consistently contributed to reduced task completion time across different study settings [Figure 14].

3.8.2. Sensitivity Analysis for Success and Ease of Understanding

The sensitivity analysis for success and ease of understanding indicated that the overall effect size was sensitive to the inclusion of individual studies. When one study was removed, the pooled standardised mean difference (SMD) shifted from 0.16 (95% CI [−0.23, 0.55], $p = 0.41$) to −0.03 (95% CI [−0.28, 0.23], $p = 0.83$). This showed that the previously observed slight positive effect was not stable. The confidence interval after removing the study became more centred around zero, and the p-value increased, indicating a further reduction in statistical significance. This finding suggests that the perceived success and ease of understanding of AI tools may vary across different study contexts, and the overall effect is not robust. The results highlight potential variability in how students engage with AI-based learning tools, possibly influenced by factors such as tool design, prior computer programming experience, or instructional integration [Figure 15].

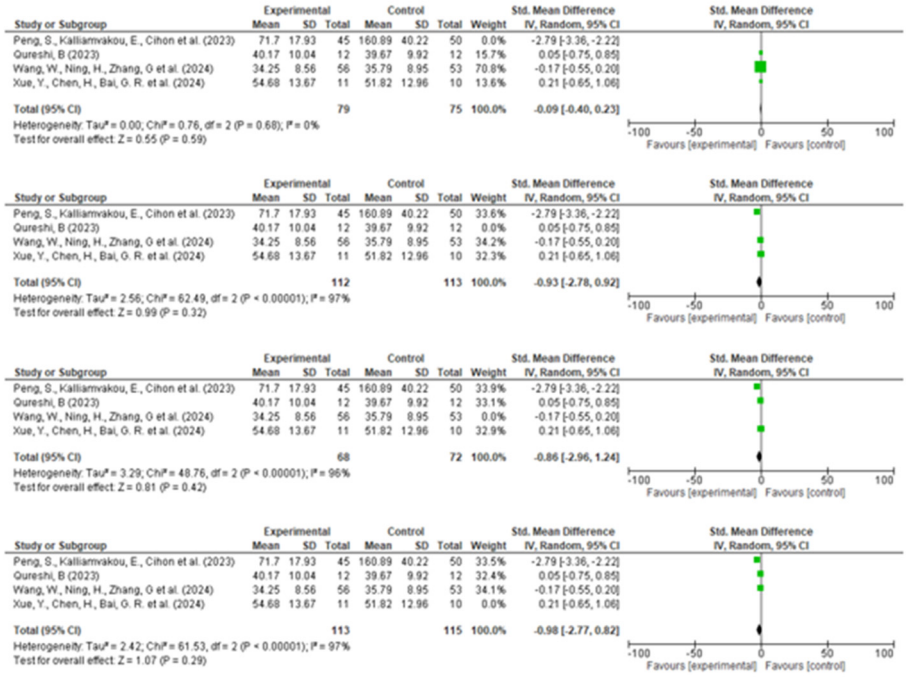

Figure 14. Leave-one-out sensitivity analysis for task completion time [17,19,23,24].

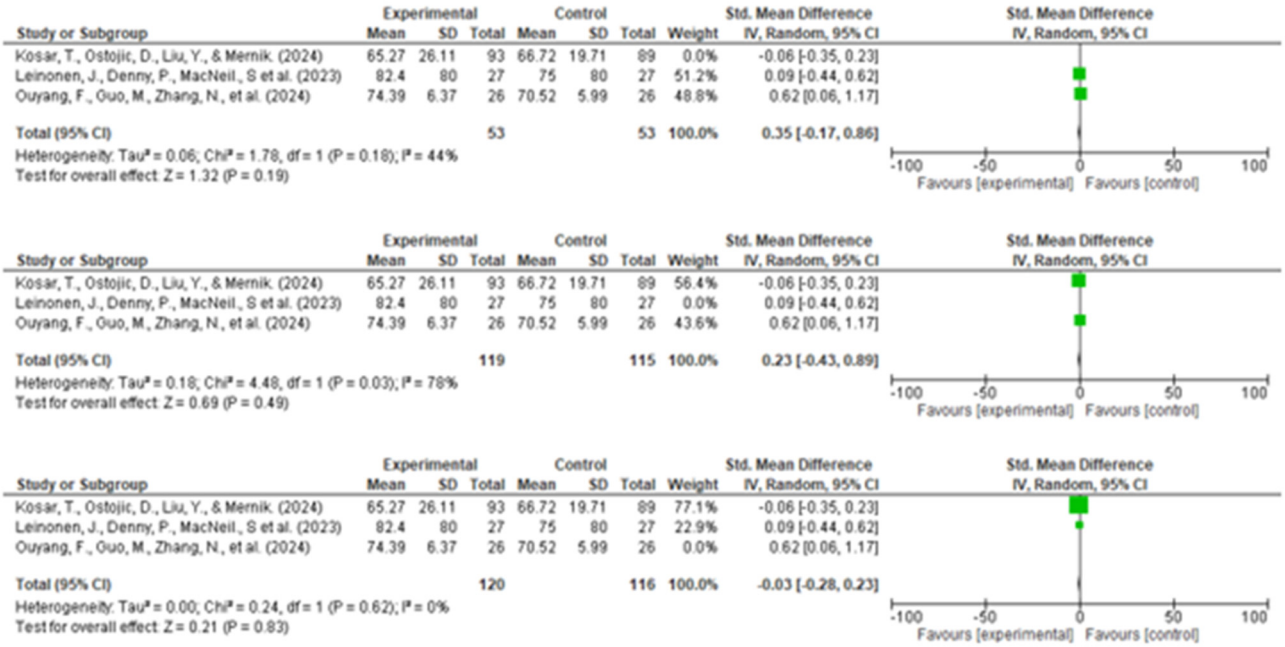

Figure 15. Leave-one-out sensitivity analysis for success and ease of understanding [20–22].

3.8.3. Sensitivity Analysis for Student Performance

The sensitivity analysis for student performance showed no significant variations among the included studies, confirming the robustness of the findings. The results remained consistent when individual studies were excluded, indicating that any single study did not unduly influence the overall conclusion [Figure 16].

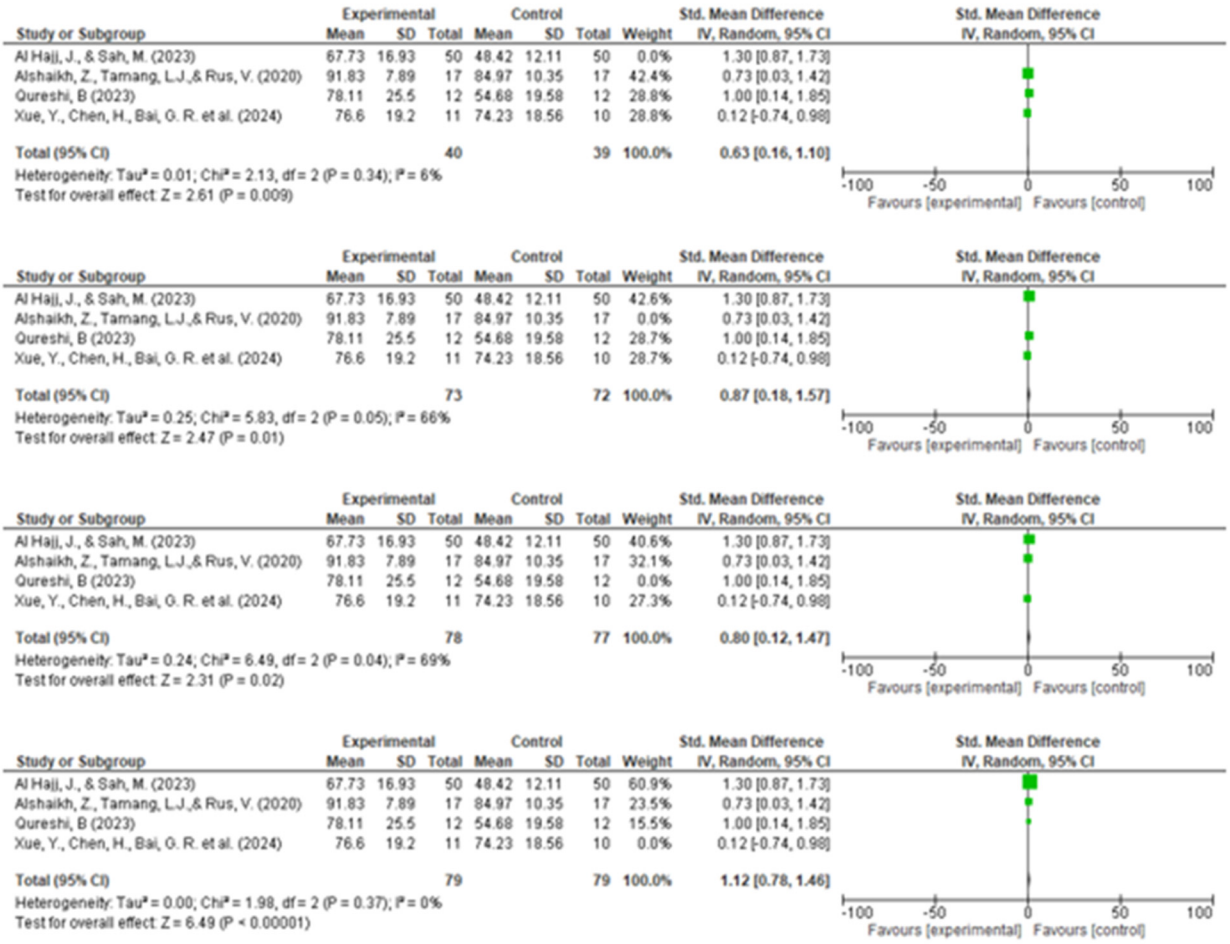

Figure 16. Leave-one-out sensitivity analysis for student performance [17,19,23,24].

4. Discussion

The findings of this systematic review and meta-analysis strongly indicate that AI tools, including ChatGPT and other computer programming assistants, have a significant positive impact on students' learning outcomes in introductory programming courses. Across 35 studies, which included randomised controlled trials and quasi-experimental designs, strong evidence supports AI tools' contributions to computer programming proficiency, task completion rates, and student engagement.

These results have profound implications for educational practice, particularly in introductory programming courses. AI tools that provide real-time feedback and problem-solving assistance create a more supportive learning environment. They also help students develop a deeper understanding of computer programming concepts, making AI-powered education a promising approach for improving student outcomes. The ability of AI tools to cater to individual learning needs is particularly valuable in large classrooms, where personalised attention is often limited [25–27]. Additionally, increased student engagement and motivation could lead to higher retention rates and long-term success in computer programming education.

Our results align with previous research demonstrating the positive effects of AI-based learning systems on academic achievement in computer programming education. Studies by [14,28] support the conclusion that AI-powered tools significantly enhance learning outcomes. Similar findings have also been reported in other STEM disciplines, where AI-driven learning interventions have led to improvements in student performance and comprehension.

However, heterogeneity among the included studies is evident, with some studies reporting only minor effects, while others demonstrate substantial improvements [29–31]. This variation in findings may be attributed to differences in AI tool functionalities, implementation quality, and student demographics [32–35]. For example, Ref. [14] found only a minor improvement, whereas Ref. [28] observed significant learning gains. These discrepancies highlight the complexity of AI integration in education and suggest that its effectiveness is likely context-dependent. The high variation found in the meta-analysis (I^2 = 95% for task completion time, I^2 = 54% for performance scores, and I^2 = 55% for success and ease of understanding) is likely caused by differences in study design, AI tool use, student characteristics, course structures, instructional methods, result measurement, and potential publication bias.

While the overall effect of AI tools on computer programming education remains positive, their impact may depend on specific contexts, such as course design, instructional approaches, and the individual learning needs of students [36–38]. These findings emphasise the importance of considering contextual variables when integrating AI tools into educational practice.

4.1. Strengths of This Study

This study provides a comprehensive synthesis of current evidence on the impact of AI tools in computer programming education. By incorporating 35 studies across different educational settings and methodologies, this meta-analysis presents a broad and detailed assessment of AI's role in improving student outcomes.

The use of both randomised controlled trials and quasi-experimental designs enhances the validity and generalisability of the findings. The analysis of multiple educational metrics, including final exam scores, task completion rates, and student engagement levels, provides a holistic view of how AI tools contribute to learning success.

Furthermore, this study highlights key factors influencing the effectiveness of AI-assisted learning, including differences in AI tool implementation, instructional strategies, and learning environments. A more detailed examination of the types of AI learning support reveals several distinct functionalities, such as syntax error correction, logic debugging assistance, natural language explanations, and intelligent scaffolding prompts. These differentiated support mechanisms directly impact students' learning experiences, with real-time error correction and context-sensitive explanations being among the most valued features. Collectively, these insights offer practical guidance for educators, policymakers, and technology developers aiming to optimise AI integration strategies in diverse educational contexts. A conceptual framework summarising the main types of AI tools, support modules, and educational scenarios is presented in Table 2 to guide future implementation efforts.

Building upon the summarised framework, the following discussion interprets the underlying educational mechanisms that may explain the observed benefits of AI-assisted learning. The findings of this review demonstrate that AI-assisted learning tools can significantly enhance student performance and engagement in programming education. This improvement may be attributed to several underlying educational mechanisms. First, AI tools that provide real-time feedback and error correction reduce cognitive load by allowing students to focus on higher-order problem-solving skills rather than low-level debugging. This aligns with Cognitive Load Theory, which emphasises the importance of minimising extraneous cognitive effort to facilitate meaningful learning. Second, the adaptive scaffolding offered by intelligent tutoring systems reflects principles from constructivist learning theory, which advocates that learners build knowledge actively with timely support. Furthermore, the availability of personalised explanations and code walkthroughs

enables differentiated instruction, allowing students with varying backgrounds and skill levels to progress at their own pace. These factors collectively explain why AI-supported educational interventions often yield greater short-term learning gains compared to traditional, non-adaptive instruction. However, the long-term effectiveness of these tools in fostering durable programming competencies remains an open question that future research must address.

Table 2. Framework of AI tool applications and educational use cases.

Category	AI Tool Types	Support Modules	Educational Scenarios
Error Identification	Syntax Checkers, Debugging Assistants	Real-time error detection, correction hints	Code debugging exercises, assignment support
Code Generation	AI Code Generators (e.g., GitHub Copilot)	Code suggestion, template generation	Assignment drafting, coding practice
Natural Language Explanation	AI Tutors, Feedback Systems	Code concept explanation, algorithm walkthroughs	Lecture support, self-study modules
Scaffolding Support	Intelligent Prompt Systems	Guided hints, stepwise solution prompts	Problem-solving practice, project guidance
Assessment Support	Auto-Grading Systems	Automated evaluation and feedback	Assignment grading, formative assessment
Skill Enhancement	Adaptive Learning Platforms	Customised learning paths based on performance	Personalised learning, remediation plans

4.2. Teaching Strategies to Mitigate Negative Effects

While AI-assisted learning can enhance student outcomes, it also presents potential risks, such as cognitive laziness, over-reliance on automated suggestions, and diminished debugging abilities. To mitigate these effects, several teaching intervention strategies were proposed in the reviewed studies. One effective approach involves structured scaffolding, where the level of AI assistance is gradually reduced over time to encourage independent problem-solving skills. Another strategy is the use of critical reflection assignments, where students are required to explain the reasoning behind AI-generated solutions, promoting deeper cognitive engagement. Additionally, instructor-mediated checkpoints, where students must periodically validate their work without AI assistance, can help maintain essential programming competencies. Incorporating such structured interventions is essential to ensure that AI tools complement, rather than replace, active learning processes.

4.3. Limitations

Despite the substantial benefits of AI tools in computer programming education, several limitations must be acknowledged to contextualise the findings and guide future research. These limitations primarily concern the generalisability of AI-assisted learning, the ethical implications of AI reliance, and the need for further research to optimise AI's role in education. The insights presented in the following subsections are drawn from the findings summarised in Table A1, which provides a detailed overview of studies examining the use of AI tools in computer programming education across various institutional settings.

4.3.1. Generalisability Beyond Controlled Experiments

Many studies included in this meta-analysis were conducted in controlled environments with predefined tasks and structured interventions [13,22,30]. While such conditions allow for rigorous evaluation of AI tools, they do not fully capture the complexities of real-world educational settings. Factors such as student motivation, instructor expertise, institutional policies, and curriculum integration significantly impact AI effectiveness

in practice. Research on self-paced learning environments [19] and hybrid AI–instructor collaborations [14] suggests that student engagement with AI varies depending on the availability of human oversight and institutional support. In addition, disparities in computer programming backgrounds, as observed in diverse classroom settings [21,39], indicate that AI effectiveness is not uniform across different student demographics.

To enhance the applicability of AI-assisted learning, future research should explore AI implementation in varied educational contexts beyond university-controlled experiments. These include K-12 education, online coding boot camps, and corporate training programs, where learners may have different levels of digital literacy and access to resources. Moreover, institutional factors, such as instructor training, digital infrastructure, and curriculum alignment, must be studied to understand their impact on AI adoption and student learning outcomes. Given that most of the existing research on AI-assisted computer programming education is concentrated in Western and Chinese university contexts [28,40], further studies should examine cultural and regional differences in AI adoption and its effects on diverse student populations.

4.3.2. Ethical Concerns About AI Reliance in Education

The increasing reliance on AI tools in computer programming education raises several ethical concerns, particularly regarding academic integrity, skill development, and algorithmic bias. One of the primary risks associated with AI-driven learning is the potential decline in independent problem-solving abilities. As students become increasingly dependent on AI-generated solutions, their ability to engage in analytical thinking and coding logic may deteriorate over time [20,41]. Ref. [22] found that students receiving AI-generated feedback spent significantly less time manually debugging their code, raising concerns about whether this approach fosters deep conceptual understanding or inadvertently diminishes cognitive engagement. Longitudinal research is needed to determine the long-term impact of AI-assisted learning on computer programming proficiency and knowledge retention.

Similarly, AI tools introduce challenges related to academic integrity and plagiarism. Several studies [39,42] suggest that AI-generated code explanations and debugging solutions may encourage passive consumption rather than active problem-solving. In some cases, students preferred AI-generated responses over peer-generated ones [21], raising concerns about whether AI is facilitating meaningful learning or enabling shortcuts that undermine educational goals. To address these concerns, AI-integrated assessment methods should be developed that require students to critically engage with AI-generated outputs rather than simply submitting them as their own. Furthermore, plagiarism detection tools must be tailored to recognise AI-generated content to maintain academic integrity in computer programming courses.

Algorithmic bias in AI-assisted learning presents another challenge, as AI-driven tutoring systems may provide inconsistent learning experiences across different student demographics [43,44]. For instance, AI-generated hints have been found to be less effective for students with lower prior computer programming knowledge, potentially widening the achievement gap. This disparity highlights the need for future research to assess bias in AI-generated feedback and develop more equitable AI-driven tutoring models. Ensuring that AI tools provide personalised and fair learning experiences for students from diverse backgrounds is essential to maximising the benefits of AI in education.

4.3.3. Focus on Short-Term Outcomes

One important limitation identified in the current body of research is the predominant focus on short-term learning outcomes, such as task performance, assignment completion

rates, and immediate assessment scores. Very few studies have systematically examined the long-term impacts of AI-assisted learning on knowledge retention, transfer of skills, or the ability to apply concepts in complex, real-world programming projects. Given the importance of developing durable competencies in programming education, future research should investigate the extent to which AI tools contribute to sustained knowledge retention and practical implementation capabilities over extended periods.

4.3.4. Learner Heterogeneity and Lack of Subgroup Analysis

One important limitation of this review is the limited exploration of learner heterogeneity and the absence of detailed subgroup comparisons across participant characteristics. The included studies often varied significantly in participants' programming backgrounds, education levels (e.g., high school, undergraduate, adult learners), and disciplinary fields (e.g., STEM versus non-STEM students), yet these differences were not systematically analysed. Moreover, most reviewed studies did not report detailed demographic data that would allow for robust subgroup analyses. Prior knowledge, academic discipline, and previous experience with coding likely moderate the effectiveness of AI-assisted learning tools. Future research should aim to collect comprehensive participant information and conduct subgroup analyses to better understand for whom, and under what conditions, AI-driven educational interventions are most effective.

5. Future Research Directions

To address the limitations outlined above, future research should focus on several key areas. Longitudinal studies are essential to assess the long-term impact of AI-assisted learning on computer programming proficiency, problem-solving approaches, and cognitive skill development. While the existing studies predominantly examine short-term performance improvements, it remains unclear whether AI-driven learning fosters lasting knowledge retention or results in dependency on automated assistance. Research should also explore how AI impacts students' confidence in their coding abilities and whether AI-assisted learners develop problem-solving strategies comparable to those trained in traditional settings.

In addition, comparative analyses of different AI-based educational tools are needed to evaluate their relative effectiveness. While some studies have compared ChatGPT to other AI tutors, such as BoilerTAI [14], comprehensive evaluations across multiple platforms—including GitHub Copilot, CodeCoach, and PythonTA 3.12—are still lacking. Large-scale comparative studies that assess usability, learning outcomes, and student engagement across different AI-driven educational tools would provide valuable insights for educators and policymakers seeking to integrate AI into computer programming curricula.

Another important research direction is the development and evaluation of adaptive AI learning systems. AI's potential to provide personalised, real-time feedback tailored to individual student needs remains underexplored. Some studies suggest that adaptive tutoring systems [28] can significantly enhance student engagement and performance, but their full integration into formal curricula presents challenges. Future research should examine how AI-driven scaffolding and personalised recommendations can be optimised to improve computer programming confidence and long-term skill development. Investigating adaptive AI's role in addressing different learning paces and styles would further contribute to its effective integration in educational settings.

Finally, human–AI collaboration in education warrants further exploration. Rather than replacing traditional instruction, AI should be integrated into hybrid learning models where students interact with both AI tutors and human instructors. Research suggests that human oversight remains essential in refining AI-generated feedback and ensuring

that students engage critically with learning materials [22]. Future studies should focus on developing frameworks for AI–instructor collaboration, exploring best practices for incorporating AI into computer programming education without diminishing the role of educators. Understanding how AI and instructors can co-design assignments to balance automated assistance with hands-on learning will be crucial in shaping the future of AI-enhanced education.

6. Conclusions

The integration of AI tools, such as ChatGPT and GitHub Copilot, has significantly enhanced learning outcomes in introductory programming courses. Our systematic review and meta-analysis provides compelling evidence that AI-powered tools contribute to higher exam scores, improved computer programming proficiency, increased task completion rates, and greater student engagement. By synthesising data from 32 controlled studies, our findings demonstrate that students who utilised AI tools consistently outperformed those relying solely on traditional instructional methods, completing tasks more efficiently and producing higher-quality code. While AI tools, on average, had a positive effect on learning outcomes, variations between studies suggest that factors such as tool design, student characteristics, and implementation quality may influence their effectiveness. Addressing these factors through optimised instructional strategies can enhance scalability and accessibility, further improving the impact of AI in computer programming education.

Despite these positive outcomes, substantial heterogeneity (I^2 = 88%) was observed, indicating that AI effectiveness depends on various contextual factors, including course design, AI functionality, and student interaction. While AI tools offer transformative potential, their integration into computer programming education requires carefully designed pedagogical strategies to maximise benefits while mitigating risks, such as over-reliance on automation and diminished problem-solving skills.

Future research should focus on optimising AI-assisted learning through adaptive models, personalised AI tutoring systems, and hybrid approaches that blend AI support with traditional instruction. In addition, addressing ethical concerns, including academic integrity and algorithmic bias, is essential to ensuring equitable and responsible AI use in education.

In conclusion, AI-driven learning tools are redefining computer programming education by offering personalised, efficient, and interactive learning experiences. However, their widespread adoption must be supported by evidence-based instructional strategies and continuous evaluation to sustain their effectiveness across diverse educational settings. By refining AI integration approaches, educators and institutions can harness AI's full potential to enhance student learning while fostering critical thinking and independent problem-solving skills.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/computers14050185/s1. Reference [45] is cited in the Supplementary Materials.

Author Contributions: Conceptualisation, M.A. and B.S.; methodology, M.A. and B.S.; software, M.A.; validation, M.A., B.S. and H.S.; formal analysis, M.A., B.S. and H.S.; investigation, M.A., B.S. and A.L.; resources, M.A., B.S. and A.L.; data curation, M.A. and H.S.; writing—original draft preparation, M.A. and B.S.; writing—review and editing, B.S., A.L. and H.S.; visualisation, M.A., B.S. and H.S.; supervision, B.S. and A.L.; project administration, M.A., B.S. and A.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

Table A1. Study Characteristics.

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
1	[46]	Pre-test–post-test quasi-experimental design.	Hacettepe University, Ankara, Turkey.	A total of 42 senior university students (28 males, 14 females), volunteers.	Pair programming in the experimental group during a six-week implementation.	Solo programming performed in the control group.	Flow experience, coding quality, and coding achievement.
2	[13]	Mixed-methods study (work-in-progress).	United States.	Participants: Introductory Java programming students at a large public university in the United States.	Use of AI tools: (1) No external help, (2) help of an AI chatbot, and (3) help of a generative AI tool, such as such as GitHub Copilot.	Condition (1): No external help.	Programming skills, students' experiences, and perceptions of AI tools
3	[47]	Mixed-methods approach (quantitative and qualitative evaluation).	University of Basilicata, Potenza, Italy.	University programming students, randomly assigned to experimental and control groups.	The experimental group used a Mixed Reality (MR) application with a conversational virtual avatar for pair programming.	Followed traditional in-person pair programming methods.	Improvement in coding skills (measured by coding assessments). User satisfaction (post-session surveys on application features, usability, and effectiveness). Addressing challenges of in-person and remote pair programming collaboration.
4	[14]	Mixed-methods approach, combining quantitative and qualitative analysis to evaluate the integration of generative AI (GenAI) in educational forums.	Purdue University. Conducted in four first- and second-year computer science courses during the Spring 2024 semester, each with substantial enrolments (~200 students per course).	Participants included teaching assistants (TAs) and approximately 800 undergraduate students in first- and second-year computer programming courses.	A generative AI platform, BoilerTAI, was used by one designated TA per course (AI-TA) to respond to student discussion board queries. AI-TAs exclusively used the platform to draft responses and provided feedback on its effectiveness.	Remaining TAs and students were unaware of BoilerTAI's use, ensuring a controlled experimental environment. Responses were presented as if written solely by the AI-TA.	Efficiency: Overall, 75% of AI-TAs reported improvements in response efficiency. Response Quality: Overall, 100% of AI-TAs noted improved quality of responses. Student Reception: Positive responses from students to AI-generated replies (~75% of AI-TAs observed favourable feedback). Human Oversight: AI-TAs required significant modifications to AI responses approximately 50% of the time. Potential for Integration: Overall, 75% of AI-TAs agreed on the potential for broader integration of BoilerTAI into educational practices.
5	[48]	Type: Mixed-methods study incorporating quantitative and qualitative data analysis. Duration: Conducted over a 12-week semester during the September 2022 offering of a CS1 course. Focus: Investigated the usage and perceptions of PythonTA, an educational static analysis tool.	University of Toronto. Conducted at a large public research-oriented institution in a first-year computer science (CS1) course.	A total of 896 students (42.5% men, 39.3% women, 1% non-binary, 17.2% undisclosed), with diverse ethnic backgrounds, enrolled in an introductory programming course (CS1).	Integration of PythonTA, a static analysis tool, in programming assignments; 10–20% of grades tied to PythonTA outputs; students encouraged to use PythonTA locally and via an autograder.	Implicit subgroups based on prior programming experience: None (Novices): No prior programming experience. Course: Formal programming education.	PythonTA usage frequency. Self-efficacy in responding to PythonTA feedback. Perceived helpfulness of PythonTA. Changes in coding practices and confidence over the semester.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
6	[28]	Controlled experiment to measure programming skills through course scores and programming task performance.	Anhui Polytechnic University, China. University setting with focus on third-year and fourth-year students.	A total of 124 third-year undergraduate students and 5 fourth-year students. Background: Students studied C and Java programming languages as part of their curriculum.	Programming tasks (five levels): Included Java and C programming tasks with varying difficulty. Tasks assessed correctness and efficiency, using EduCoder platform for evaluation. Efficiency score calculated based on task completion time and accuracy.	Senior students served as a comparative baseline. Their performance on tasks 3 and 4 was compared with that of juniors to evaluate skill differences.	Correlation between course scores and programming skills. Identification of courses significantly related to programming skills (e.g., software engineering). Analysis of task completion rates, correctness, and efficiency scores. Comparison of junior and senior programming performance.
7	[15]	The study introduced CodeCoach, a programming assistance tool developed to enhance the teaching and learning experience in programming education. It focused on addressing coding challenges faced by students in constrained lab environments.	Virtual lab environment. Sri Lanka Institute of Information Technology (SLIIT) in Malabe, Sri Lanka.	A total of 15 students and 2 instructors who participated in a virtual lab session. The participants' performance and the tool's effectiveness were assessed during the session.	The primary intervention was the CodeCoach tool. The tool utilised the GPT-3.5 AI model to provide tailored hints and resolve programming errors. Key features included community forums, lab management capabilities, support for 38 programming languages, and automated evaluation for coding challenges.	The research compared outcomes with traditional instructional methods (e.g., direct instructor feedback and error correction without the tool). It highlighted the limitations of non-automated feedback methods due to time constraints and lack of scalability. The other performance with the tool was implicitly compared to traditional learning methods without such tools.	Effectiveness of the tool in enhancing programming learning. Student and instructor satisfaction with the tool. Number of AI-generated hints used by students. Individual student progress during coding tasks. Improvement in logical thinking and problem-solving skills related to programming
8	[21]	The study employed a comparative experimental design to evaluate code explanations generated by students and those generated by GPT-3. The primary focus was on three factors: accuracy, understandability, and length. The study involved two lab sessions and integrated thematic and statistical analyses.	First-year programming course at The University of Auckland, with data collected during lab sessions over a one-week period.	Participants: Approximately 1000 first-year programming students enrolled in the course. Experience Level: Novices in programming. Language Covered: C programming language. Tasks Assigned: Creating and evaluating code explanations for provided function definitions.	Treatment Group: Exposed to GPT-3-generated code explanations during Lab B. Content: Students evaluated and rated explanations (both GPT-3 and student-created) on accuracy, understandability, and length. Prompts: Students were also asked to provide open-ended responses on the characteristics of useful code explanations.	Comparison Group: Student-generated code explanations from Lab A, created without AI assistance. Evaluation: Students rated both their peers' and GPT-3's explanations to enable comparison.	Accuracy: How well the explanation described the code's functionality. Understandability: Ease of comprehending the explanation. Length: Whether the explanation was considered ideal in length. Thematic Analysis: Students' perceptions of what makes a good explanation (e.g., detailed, line-by-line descriptions). Effect Sizes: Differences in quality metrics (using Mann–Whitney U tests) between student and GPT-3 explanations. Preferences: Students preferred GPT-3 explanations for their clarity and precision.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
9	[49]	A comparative design was used to assess the quality of code explanations created by students versus those generated by GPT-3. It involved a two-phase data collection process (Lab A and Lab B), with students evaluating explanations based on their accuracy, understandability, and length.	The University of Auckland, specifically in a first-year programming course.	Participants: Approximately 1000 novice programming students. Experience Level: Beginner-level programming skills. Focus Area: Understanding and explaining code in the C programming language.	The students in Lab B were presented with code explanations generated by GPT-3 alongside explanations created by their peers from Lab A. The students evaluated the explanations based on criteria such as accuracy, understandability, and length. The GPT-3-generated explanations were designed to serve as an example to scaffold the students' ability to explain code effectively.	The control group consisted of student-generated code explanations from Lab A. These explanations were rated and compared against those generated by GPT-3 in Lab B.	Accuracy: The correctness of the explanation in describing the code's purpose and functionality. Understandability: The ease with which students could comprehend the explanation. Length: Whether the explanation was of an appropriate length for understanding. Student Preferences: Insights into what students value in a "good" code explanation (e.g., clarity, detail, structure). Quantitative Metrics: Statistically significant differences were observed, showing that GPT-3 explanations were rated higher for accuracy and understandability but had similar ratings for length compared to student explanations.
10	[22]	The study employed a quasi-experimental design to evaluate the effects of ChatGPT intelligent feedback compared to instructor manual feedback on students' collaborative programming performance, behaviours, and perceptions. The research combined quantitative and qualitative methods, including learning analytics approaches, thematic analysis, and performance assessment, to investigate the cognitive, regulative, and behavioural aspects of the intervention.	A graduate-level face-to-face course titled "Smart Marine Metastructure" at a top university in China during the summer of 2023.	Participants: A total of 55 graduate students, including 13 doctoral and 42 master students. Grouping: The students were arranged into 27 groups (2–3 students per group) based on their pre-course programming knowledge scores, balancing groups with higher and lower scorers. Programming Context: The course emphasised advanced applications of artificial intelligence in ocean engineering, using Matlab for collaborative programming tasks.	Control Group: A total of 14 groups received instructor manual feedback, which provided text-based evaluations in five dimensions: correctness, readability, efficiency, maintainability, and compliance. Experimental Group: A total of 13 groups received ChatGPT intelligent feedback, delivered as both textual and video feedback created by a virtual character. Feedback Mechanism: ChatGPT was prompted to assess Matlab code and generate specific suggestions for improvement across five dimensions.	The control group received manual feedback from the instructor, which included the following: Detailed comments on code. Evaluations across the same five dimensions as ChatGPT feedback. Text-based explanations to facilitate cognitive collaboration.	Performance Metrics: Post-course programming knowledge (assessed out of 100 points). Quality of group-level programming products (evaluated by the instructor). Collaborative Programming Behaviours: Analysed using learning analytics (e.g., clickstream analysis, epistemic network analysis). Focused on cognitive-oriented discourse (control) and regulation-oriented discourse (experimental). Perceptions of Feedback: Students' self-reported strengths and weaknesses of each feedback type, gathered through thematic analysis. Behavioural Analysis: Frequency and patterns of collaborative actions (e.g., programming exploration, task monitoring, code writing).

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
11	[36]	A controlled, between-subjects experiment was used to assess the impact of ChatGPT usage on the learning outcomes of first-year undergraduate computer science students during an Object-Oriented Programming course. Students were divided into two groups: one encouraged to use ChatGPT (treatment group) and the other discouraged from using it (control group). Performance on lab assignments, midterm exams, and overall course grades was evaluated using statistical analysis.	University of Maribor, Faculty of Electrical Engineering and Computer Science (FERI), Slovenia, during the spring semester of 2023.	Participants: A total of 182 first-year undergraduate computer science students, aged approximately 19.5 years, with 85.9% identifying as male, 12.4% as female, and 1.7% preferring not to say. Division: Randomly divided into two groups, Group I (ChatGPT users) and Group II (non-ChatGPT users), each initially containing 99 participants.	Group I (Treatment): Students were allowed and encouraged to use ChatGPT for practical programming assignments. Assignments were adjusted to reduce reliance on ChatGPT, including modifications such as minimal textual instructions, provided UML diagrams, and extension tasks requiring independent effort during lab sessions. Defences: Students defended their assignments in lab sessions through interactive questioning, ensuring comprehension and reducing reliance on ChatGPT. Exams: Paper-based midterm exams assessed Object-Oriented Programming knowledge without allowing ChatGPT or other programming aids. Feedback Questionnaires: Weekly and final feedback questionnaires collected data on ChatGPT usage, assignment complexity, and student perceptions.	Group II (Control): Students were instructed not to use ChatGPT for assignments. Weekly questionnaires confirmed adherence, and eight participants who reported ChatGPT usage were excluded from the analysis. Assignments: The same assignments as Group I, with adjustments to prevent reliance on AI-based solutions.	Lab assignment performance was assessed weekly, with success rates calculated as percentages of completed mandatory and optional assignments. Results: No statistically significant difference in lab performance between Group I (65.27%) and Group II (66.72%). Midterm Exam Results: Two paper-based exams assessing theoretical and practical programming knowledge. Results: No significant difference between Group I and Group II (Group I: 65.96%, Group II: 66.58%). Overall Course Grades: Combination of midterm exam scores (50%) and lab assignments (50%). Results: No statistically significant difference in overall success between Group I (65.93%) and Group II (66.61%). Student Perceptions: Group I participants reported benefits, such as program understanding and code optimisation, but noted limitations, such as occasional inaccuracies and reduced learning engagement. Usage Trends: Group I students primarily used ChatGPT for code optimisation and comparison rather than direct code generation.
12	[19]	The study employed a controlled experimental A/B design to examine the impact of ChatGPT 3.5 on CS1 (introductory computer science) student learning outcomes, behaviours, and resource utilisation. Participants were divided into two groups: experimental (with ChatGPT access) and control (without ChatGPT access).	The experiment was conducted in Spring 2023 at a private, research-intensive university in North America offering a Java-based CS1 course.	Initially, 56 students were recruited, but 48 participants submitted valid screen recordings. Experimental Group: Twenty-three students. Control Group: Thirty-three students. Demographics: The groups were balanced in terms of Java programming experience and midterm scores (Mann–Whitney U-tests showed no significant differences). Tasks: The students designed UML diagrams, implemented Java programming tasks, and completed a closed-book post-evaluation.	Experimental Group: Access to ChatGPT 3.5 and other online resources (e.g., Google, Stack Overflow). Allowed to consult ChatGPT for task completion but were not obligated to use it. Tasks: UML Diagram: Create class structures, including relationships, fields, and methods. Java Programming: Implement class skeletons based on UML diagrams. Post-Evaluation: Answer conceptual and coding-based questions testing Object-Oriented Programming (OOP) principles.	Control Group: Access to all online resources except ChatGPT. Similar tasks and requirements as the experimental group, ensuring consistency.	Learning Outcomes: Graded on UML diagrams (40 points), programming tasks (40 points), and post-evaluation quizzes (8 points). Time spent on tasks (UML design, programming, post-evaluation) was also recorded. No statistically significant differences in performance or completion time were observed between groups. Resource Utilisation: ChatGPT Group: Relied predominantly on ChatGPT, with minimal use of traditional resources (e.g., lecture slides, course materials, or Piazza). Non-ChatGPT Group: Utilised a broader range of educational resources. Perceptions: The post-survey captured attitudes toward ChatGPT's utility, ethical concerns, and reliability.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
					Tasks were completed independently, and participants recorded their screen activities.		Most participants exhibited neutral or slightly positive attitudes, with significant concerns about over-reliance and ethical implications.
13	[50]	The study used a quasi-experimental design with two groups to evaluate the impact of automatically generated visual next-step hints compared to textual feedback alone in a block-based programming environment (Scratch). The study aimed to assess the effects of these hints on students' motivation, progression, help-seeking behaviour, and comprehension.	The study was conducted in Bavaria, Germany, at a secondary school with two seventh-grade classes. Both classes were taught Scratch programming by the same teacher.	A total of 41 students aged 12–13 participated. Two cohorts: One class (19 students) received next-step visual hints (treatment group), and the other (22 students) received only textual feedback (control group). Four participants were excluded due to technical issues, leaving 37 valid cases (15 in the treatment group and 22 in the control group).	Treatment Group: Received textual feedback about failing tests and visual next-step hints generated by the Catnip system, suggesting specific code changes (e.g., adding, deleting, or moving blocks). Hints were shown automatically after a failed test when students clicked the "Test" button. Task: Students followed a Scratch tutorial to create a simple game involving navigating a boat to a goal without hitting walls. Tutorial steps included implementing functionalities such as player movement and collision detection. The activity lasted 25 min, followed by a post-test survey and comprehension tasks.	Received only textual feedback describing which tests had failed, without visual hints or next-step guidance. The tasks, setup, and duration were identical to the treatment group, ensuring comparability.	Motivation: Measured using a five-point Likert scale in a post-test survey. Focused on students' enjoyment and confidence in completing the tasks. Progression: Tracked by the number of tutorial steps completed over time. Analysed using automated tests to determine whether students passed specific tutorial milestones. Help-Seeking Behaviour: Help-seeking behaviour was measured by the following: The number of clicks on the "Test" button (indicating hint usage). The number of help requests directed to teachers. Comprehension: Evaluated through three post-task comprehension questions requiring students to identify correct code segments from provided options.
14	[39]	The study employed a descriptive and exploratory design to evaluate the impact of a permissive policy allowing students to use ChatGPT and other AI tools for programming assignments in CS1 and CS2 courses. The research focused on understanding how students utilised these tools, their learning outcomes, and perceptions. Surveys, reflective learning forms, and coding scheme analysis were utilised to gather data on students' behaviour and attitudes toward AI tools.	The study was conducted at a private Midwestern university in the United States during the Spring 2023 semester.	Students: Enrolled in one CS1 and one CS2 section. Participants included students at varying levels of familiarity with AI tools. Sample Size: A total of 40 learning reflections were submitted by the students across multiple assignments. Pre- and post-semester surveys captured student perspectives on AI use. Demographics: Participants' attitudes and experiences with ChatGPT varied, allowing the study to capture a diverse range of perspectives.	AI Permissive Policy: Students could use ChatGPT and similar tools freely for assignments. Requirement: Students were required to submit a reflective learning form documenting the following: AI chat transcripts. Use of AI-generated content in submissions. Reflections on what they learned from AI interactions. Survey Data: Pre- and post-semester surveys gathered data on students' familiarity with AI, their use cases, and perceptions of academic honesty and policy.	The study did not include a traditional control group with strict AI restrictions. Instead, the analysis compared the following: Student submissions that adhered to the policy (e.g., reflective forms completed) versus those that did not. Pre-semester attitudes with post-semester changes to assess shifts in perception and behaviour.	AI Usage Patterns: Types of questions posed to ChatGPT (e.g., debugging vs. seeking full solutions). How students integrated AI-generated content into their submissions. Learning Evidence: Analysed student reflections for understanding of programming concepts. Identified cases where AI interactions led to meaningful learning versus over-reliance on AI. Student Attitudes: Pre- and post-semester surveys captured changes in how students viewed AI tools (e.g., "nervous or scared" vs. "excited"). Opinions on institutional AI policies. Challenges Identified: Cases of improper use or over-reliance on AI. Instances where AI solutions hindered learning (e.g., solving problems above students' skill levels).

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
					Assignment Guidelines: Students could request full or partial solutions from ChatGPT, debug their code, or seek help with isolated programming concepts. Students were encouraged to reflect on learning rather than rely entirely on AI-generated solutions.		Recommendations: Strategies for promoting responsible and effective AI use in education, including prompt engineering, reflection incentives, and transparency mechanisms.
15	[18]	Type: Controlled 2 × 2 between-subject experiment. Tasks: Two primary tasks: Coding Puzzles: Solve coding problems of moderate difficulty on an online platform with automated judging. Typical Development Task: Fix two bugs in a small Python project.	Conducted at a mid-size public university in Beijing, China, focusing on Information and Communication Technology (ICT).	Total Participants: A total of 109 computing majors. Demographics: A total of 89 males and 20 females. Aged 20–26 years. Education Level: A total of 7 undergraduates, 102 postgraduate students. Experience: At least one year of professional software development, mostly as interns in major companies (e.g., Baidu, Tencent, Microsoft). The participants were randomly assigned to one of four groups.	Experiment Setup: Participants in the intervention groups had access to ChatGPT (GPT-3.5) for task assistance. Tasks were completed in environments supporting code writing and debugging. ChatGPT accounts and internet access were provided for groups using the tool. Procedure: Tasks were completed within a 75 min timeframe. Interaction with ChatGPT was logged and analysed for insights.	Participants in the control groups completed tasks without ChatGPT assistance but could use the internet as a resource.	Efficiency: Time taken to complete tasks. Solution Quality: Assessed using a set of test cases. Subjective Perception: Measured through post-experiment surveys (e.g., perceived utility of ChatGPT). Task Load: Evaluated using NASA-TLX scales for workload. Interaction Patterns: Analysed from ChatGPT logs for insights into collaboration dynamics.
16	[24]	Pilot study using a pre-test–post-test design. Purpose: To evaluate the effectiveness of the Socratic Tutor (S.T.), an intelligent tutoring system, in improving programming comprehension and self-efficacy among novice programmers.	Conducted in a computer lab at an urban university in Southeast Asia.	Participants: A total of 34 computer science students enrolled in introductory programming courses. Demographics: Background questionnaire and self-efficacy survey completed by all participants. Knowledge levels: Participants divided into two groups based on pre-test scores (TOP and BOTTOM).	Tutoring System: Socratic Tutor (S.T.), a dialogue-based ITS inspired by the Socratic method. Features: Programming language independence, scaffolding through guided questions, three-level feedback system. Session: A 60 min tutoring session with nine Java code examples. Tasks: Students analysed code, predicted outputs, and engaged in Socratic dialogues to address misconceptions. Feedback levels: Level 1: Conceptual explanation. Level 2: Fill-in-the-blank hints. Level 3: Multiple-choice question hints.	No explicit control group was used in the pilot study, as all participants received the same intervention. Pre-test scores served as a baseline for measuring improvements.	Learning Gains (LGs): Computed based on pre-test and post-test scores (each with nine Java programs requiring output prediction). An average improvement of 12.58% (from 75.82% pre-test to 88.4% post-test; LG score: 52.03%). Greater gains observed in the BOTTOM group (lower prior knowledge) compared to the TOP group (higher prior knowledge). Self-Efficacy: Measured via an 11-item survey on programming concepts. Participants with higher self-efficacy showed slight, non-statistically significant learning advantages. Feedback Effectiveness: Success rates for each feedback level: Level 1: 62.02% correct responses. Level 2: Increased by 14.45%. Level 3: Increased by 7.03%.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
17	[51]	A controlled experiment aimed at quantifying and comparing the impact of manual and automated feedback on programming assignments. It involved three distinct conditions: feedback from teaching assistants (TAs) only; feedback from an Automated Assessment Tool (AAT) only; and feedback from both TAs and the AAT. The study evaluated these conditions based on objective task effectiveness and subjective student perspectives.	The experiment was conducted in the Bachelor of Software Development program at the IT University of Copenhagen (ITU), Denmark, during the first semester of 2022.	Participants: A total of 117 undergraduate first-semester students (20% women). Programming Background: Overall, 33% had little or no prior experience; 45% had limited experience.; and 22% had prior programming experience. Context: The participants were enrolled in an introductory programming course (CS1).	Programming Task: Solve a modified "FizzBuzz" problem, requiring nested if-else statements and Boolean conditions. Feedback Conditions: TAs Only—Students had access to formative feedback from qualified teaching assistants. AAT Only—Students received summative pass/fail feedback via the Kattis platform.TAs + AAT—Students could access both formative (TAs) and summative (AAT) feedback. Duration: A total of 1 h (with an additional 10 min, if needed). Metrics Recorded: Objective—Correctness (unit test results) and task duration, code smells (via SonarCube). Subjective—Student-reported frustration, unmet assistance needs, and feedback preferences.	Each feedback condition served as a control for the others. Random assignment to conditions ensured balance: TAs Only—39 students.; AAT Only—42 students.; and TAs + AAT—36 students.	Objective Metrics Correctness: Percentage of successful unit test cases. Students with both TAs and AAT feedback performed significantly better ($p = 0.0024$). Duration: Time spent on the assignment decreased from TAs Only to TAs + AAT ($p = 0.028$). Code Smells: Fewer code smells were found in solutions from the TAs + AAT condition compared to the TAs Only condition ($p = 0.015$). Subjective Metrics Frustration: Women reported higher frustration overall, particularly under the AAT Only condition ($p = 0.068$). Unmet Assistance Needs: Women in the AAT Only condition reported significantly more unmet assistance needs compared to the TAs or TAs + AAT conditions ($p = 0.0083$). Preferences: Women preferred TAs over AAT for feedback ($p = 0.0023$), while men exhibited no clear preference.
18	[44]	The study used a mixed-methods design, integrating quantitative experiments and qualitative thematic analysis to assess the impact of prompt-tuned generative AI (GenAI) conversational agents on computational thinking (CT) learning outcomes and usability. It used an ABBA experimental design to compare control and intervention phases, as well as reflection reports for in-depth insights into long-term use.	The study took place within a software design class during the fall semester of 2023 at a Swiss university.	Participants: A total of 23 undergraduate students were initially enrolled; 21 completed the study. Demographics: A total of 8 females and 15 males. Background: Students in their third year of bachelor's studies in business and economics, with varying programming experience: 17 students reported prior programming knowledge; 2 reported no experience; 2 did not respond. Previous Interaction with ChatGPT: Average of 6.11 months of use prior to the course.	Participants: A total of 23 undergraduate students were initially enrolled; 21 completed the study. Demographics: A total of 8 females and 15 males. Background: Students in their third year of bachelor's studies in business and economics, with varying programming experience: 17 students reported prior programming knowledge; 2 reported no experience; 2 did not respond. Previous Interaction with ChatGPT: Average of 6.11 months of use prior to the course.	Condition A (control): Labs 1 and 4 with a default, non-configured Graasp Bot. Condition B (intervention): Labs 2 and 3 with the CT-prompt-tuned Graasp Bot.	Usability Metrics (seven-point Likert scale): Usefulness. Ease of Use. Learning Assistance. Learning Outcomes: Lab assignment scores, normalised to a 100-point scale. Accuracy rates for exercises attempted. Attitudes Towards Chatbots: Measured using the General Attitude Towards Robots Scale (GAToRS) before and after the study. Reflection Reports: Student perceptions of strengths and limitations of ChatGPT and Graasp Bot. Interaction Logs: Number and nature of interactions with ChatGPT and Graasp Bot.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
19	[43]	Mixed-methods, large-scale, controlled study with 120 participants.	Three academic institutions in the United States: Northeastern University (R1 University), Oberlin College (Liberal Arts College), and Wellesley College (Women's College).	The participants were university students who had completed a single introductory computer science course (CS1). The population included first-generation students, domestic and international students, and participants from diverse academic and demographic backgrounds.	Participants interacted with Codex via a web application called "Charlie the Coding Cow". Tasks included crafting natural language prompts for 48 problems divided into 8 categories. Problems were designed at the CS1 skill level, and correctness was tested automatically.	There was no traditional control group; all participants interacted with the intervention (Charlie). Comparisons were made within the group based on variables such as prior experience, demographics, and problem difficulty.	Success rate: Fraction of successful attempts at solving problems. Eventual success rate: Final success after multiple attempts. Pass@1: Probability of success from prompt sampling. Surveys and interviews captured perceptions and strategies.
20	[41]	Between-subjects design. The experimental group used ChatGPT exclusively, while the control group used other resources except genAI tools.	Software engineering courses and universities EPIC Lab.	Undergraduate software engineering students ($N = 22$) with low to medium familiarity with Git, GitHub, and Python.	ChatGPT was used for completing three tasks related to software engineering (debugging, removing code smells, and contributing to GitHub).	Participants in the control group used any online resources except genAI tools for the same tasks.	Productivity (task correctness), self-efficacy, cognitive load (NASA TLX), frustration, and participants' perceptions of ChatGPT's faults and interactions.
21	[52]	Type: Survey-based research using quantitative and qualitative methods. Focus: To understand students' use patterns and perceptions of ChatGPT in the context of introductory programming exercises. Structure: Students completed programming tasks with ChatGPT-3.5 and then responded to an online survey regarding their usage patterns and perceptions. Research Questions: What do students report on their use patterns of ChatGPT in the context of introductory programming exercises? How do students perceive ChatGPT in the context of introductory programming exercises?	Location: Goethe University Frankfurt, Germany. Timeframe: Winter term 2023/24 (starting from 6 December 2023).	Sample Size: A total of 298 computing students enrolled in an introductory programming course. Demographics: The majority were novice programmers: 34% had no programming experience, 43% had less than one year, 17% had 1–2 years, and 6% had over three years of experience. The majority had prior experience using ChatGPT (84%).	Task Structure: Students completed a newly designed exercise sheet comprising tasks involving recursion, lists, functions, and conditionals. Tasks required interpreting recursive code, solving algorithmic problems, and generating code with optional recursion. Tool: ChatGPT-3.5, accessed via the free version. Instructions: The students used ChatGPT independently without structured guidance, except for a link to OpenAI's guide on prompt engineering. They were asked to record all prompts and responses as paired entries for submission.	Not Applicable: The study did not include a formal control group for comparison, as all students in the course used ChatGPT-3.5 as part of the task.	Use Patterns: Frequency, duration, and purpose of ChatGPT usage during the programming exercises. Common use cases included problem understanding, debugging, and generating documentation. Perceptions: Students' evaluations of ChatGPT's ease of use, accuracy, relevance, and effectiveness. Analysis of positive and negative experiences through Likert-scale responses and open-ended survey answers. Challenges Identified: Over-reliance, inaccuracies in AI-generated responses, and the need for critical engagement. Pedagogical Implications: Insights for educators on integrating and guiding the use of GenAI tools in programming education.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
22	[53]	Mixed-methods research combining thematic analysis and quantitative analysis. Focus: The study explored how novice programmers aged 10–17 use and interact with AI code generators, such as OpenAI Codex, while learning Python programming in a self-paced online environment. Structure: Ten 90 min sessions held over three weeks, including a pre-test, seven Python training sessions, and two evaluation sessions (immediate post-test and retention post-test).	Participants were recruited from coding camps in two major North American cities and attended the sessions remotely via Google Meet.	Sample Size: A total of 33 novice learners in the experimental group (out of 69 in the original study). Age Range: Aged 10–17 years old (mean age = 12.7, SD = 1.9). Demographics: A total of 11 females and 22 males. A total of 25 participants were English speakers. Experience: None of the participants had prior experience with text-based programming. A total of 32 participants had experience with block-based programming (e.g., Scratch). A total of 12 participants had attended a programming-related class previously.	Tool: AI code generator based on OpenAI Codex embedded in the Coding Steps IDE. Environment: Self-paced online Python learning platform providing the following: A total of 45 programming tasks with increasing difficulty. AI code generator for generating solutions. Real-time feedback from remote instructors. Python documentation, worked examples, and debugging strategies. Tasks: Split into code-authoring and code-modification parts. Support Mechanisms: Learners could prompt the AI generator for code. No AI access was provided for code-modification tasks.	Condition: Participants in the control group had no access to the AI code generator and completed the tasks manually using only the provided learning materials and instructor feedback.	Learning Outcomes: Retention post-test scores (one week after the study). Immediate post-test scores. Both tests included the following: 10 coding tasks (split into code-authoring and code-modification); 40 multiple-choice questions on programming concepts. No AI access was allowed during the evaluation phase. Behavioural Analysis: Frequency and context of AI code generator use. Coding approaches (e.g., AI Single Prompt, AI Step-by-Step, Hybrid, Manual). Prompt crafting patterns and language used. Code Quality: Properties of AI-generated code (correctness, complexity, alignment with curriculum). Utilisation patterns of AI-generated code (e.g., verification, placement, modification). Self-Regulation and Over-Reliance: Instances of tinkering and verification by learners. Patterns of over-reliance on AI-generated solutions.
23	[54]	Mixed-methods research combining formative studies, in-lab user evaluations, and computational assessments. Objective: To evaluate the effectiveness of HypoCompass in training novice programmers on hypothesis construction for debugging, using explicit and scaffolded instructions.	Conducted in a private research institution in the United States.	Main Study: A total of 12 undergraduate and graduate students with basic Python programming knowledge but limited expertise. Screening Survey: Of 28 students, 12 were selected for the study. Background: The average age was 22.5, with nine females, three males, and seven non-native English speakers. Pilot Studies: Conducted with eight additional students.	Tool: HypoCompass, an LLM-augmented interactive tutoring system designed to complete the following: Simulate debugging tasks with buggy codes generated by LLMs. Facilitate hypothesis construction through role-playing as teaching assistants (TAs). Provide feedback and scaffolding via hints, explanation pools, and code fixes. Process: Participants completed pre- and post-tests designed to evaluate their debugging skills. Interaction involved debugging two programming exercises with three buggy codes each.	Preliminary Control Conditions: Control—LLM: Practice materials generated by LLMs without HypoCompass. Control—Conventional: Traditional debugging exercises from a CS1 course. Preliminary results suggested HypoCompass outperformed the controls in learning gains, but larger-scale studies were planned.	Quantitative Metrics: Pre- to post-test performance: Improvement: Significant 16.79% increase in debugging accuracy. Efficiency: A 12.82% reduction in task completion time. Hypothesis construction: Comprehensive (LO1): Marginal 2.50% improvement; 26.05% time reduction. Accurate (LO2): Significant 27.50% improvement; 7.09% time reduction. Qualitative Feedback: Students found HypoCompass engaging, helpful, and motivating for debugging. Feedback highlighted the usefulness of scaffolding, hint systems, and interactive explanations. Concerns included the potential for over-reliance on scaffolding and UI preferences for natural coding environments. Instructor Insights: HypoCompass could supplement CS1 curricula, TA training, or debugging tutorials.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
					Tasks included creating test suites, hypothesising about bugs, and revising faulty code. Duration: Each session lasted ~1 h, including pre-survey, pre-test, HypoCompass interaction, post-test, and post-survey.		Suggestions included modularising features for easier classroom integration and enabling use in standard coding IDEs.
24	[55]	Mixed-methods research incorporating quantitative user studies (controlled experiments) and qualitative surveys and interviews.	Conducted within a university setting, involving undergraduate computer science courses, specifically first-year and third-year programming and software engineering students.	First-year programming students learning foundational coding skills. Third-year software engineering students contributing to the development of an intelligent tutoring system (ITS). Tutors involved in grading and feedback.	An ITS integrating automated program repair (APR) and error localisation techniques was deployed. First-year students used ITS for debugging assistance and feedback on programming assignments. Third-year students incrementally developed and enhanced ITS components as part of their software engineering projects.	Group B (control): First-year students completed programming tasks using conventional tools without access to an ITS. Compared against Group A, who received ITS feedback and guidance during tasks.	For first-year students: Performance metrics (number of attempts, success rates, rectification rates, and rectification time for solving programming tasks). Feedback satisfaction and usefulness from surveys. For tutors: Usability and satisfaction with error localisation and feedback tools, alongside grading support. For third-year students: Experience and skill development from contributing to ITS projects.
25	[56]	The study employed a controlled experimental design comparing two groups: an experimental group with access to GPT-generated hints and a control group without access to GPT-generated hints.	The study was conducted at the Warsaw University of Life Sciences (Poland), specifically within an Object-Oriented Programming course.	The participants were second-semester computer science students enrolled in an Object-Oriented Programming course. Out of 174 students in the course, 132 students consented to participate. These students were familiar with the RunCode platform, as it had been used in a previous semester's programming course. The control group contained 66 students. The experimental group contained 66 students. A pre-test established that the two groups had no significant differences in baseline knowledge.	The experimental group received GPT-generated hints via the GPT-3.5 API integrated into the RunCode platform for 38 out of 46 programming assignments. These hints provided explanations of errors, debugging tips, and suggestions for code improvement. The GPT feedback was dynamically generated in Polish and tailored to the submitted code, compiler errors, runtime errors, or unit test failures. The hints emphasised meaningful insights without revealing the correct code solution. The experimental group rated the usefulness of the GPT hints on a five-point Likert scale.	The control group had access only to the platform's regular feedback, which included details about compiler errors, runtime errors, and unit test results. The control group did not receive GPT-generated hints and relied on standard feedback to resolve issues in their submissions.	Immediate performance: Percentage of successful submissions across consecutive attempts. Learning efficiency: Time taken to solve assignments. Reliance on feedback: Usage of the platform's regular feedback (non-GPT) for tasks with and without GPT-generated hints. Affective state: Emotional states (e.g., focused, frustrated, bored) reported during task completion. Impact of GPT feedback absence: Performance on tasks without GPT-generated hints after prior exposure to GPT-enabled tasks. User satisfaction: Perceived usefulness of GPT-generated hints (rated on a Likert scale).

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
26	[57]	The study used a correlational and exploratory experimental design. It utilised surveys and statistical analyses (e.g., Spearman correlations and stepwise regression) to identify significant factors that influence novice programmers' programming ability.	The study was conducted in a university setting, specifically within the School of Computer and Information. The institution was not explicitly named but appeared to focus on undergraduate computer science education.	The participants were undergraduate students (referred to as novice programmers) who had been learning programming for approximately three years. A total of 104 subjects participated in the study. These participants were selected based on their enrolment in programming-related courses, such as C, C++, Java, Data Structure, Java Web, and Python.	The researchers designed and administered a questionnaire to measure programming ability based on factors such as project experience, interest, and use of programming-related websites. The students' course scores from programming-related classes were collected to serve as a benchmark for programming ability. The study used statistical techniques, such as Spearman correlation and stepwise regression, to analyse the relationship between questionnaire responses and course performance. The variables identified as significant (e.g., the number of modules in a project and the number of programming-related websites visited) were validated through a symposium with experts and students.	The study did not employ a traditional control group but instead used a comparative analysis approach. Questionnaire responses were compared to course scores to identify relationships. A list of top-performing students, based on expert evaluations during the symposium, was used to validate the findings.	Significant factors influencing programming ability: Number of project modules (p.NumModule). Number of programming-related websites visited (s.NumSites). Correlation between course scores and questionnaire responses: Identified significant correlations, e.g., between course averages and project experience. Validation of findings: Comparison of top-performing students (based on expert lists) with results from a regression model. Development of a regression model: Created a predictive formula for evaluating programming ability using the two identified indicators. Practical recommendations: The study provided insights for improving educational programs and recruitment strategies based on programming ability metrics.
27	[58]	The study utilised an observational design combining data from four experiments. It focused on investigating whether self-rated experience and confidence are reliable predictors of students' performance in model comprehension tasks.	The study was conducted in university settings during requirements engineering courses offered to undergraduate and graduate students at a German university. The participants engaged in online experiments derived from industrial case studies.	Sample Size: A total of 368 participants (119 undergraduates and 249 graduates across experiments). Undergraduate Students: Enrolled in bachelor-level courses, primarily in systems engineering, information systems, or business studies. Graduate Students: Enrolled in master's-level courses, holding bachelor's degrees in related fields.	Participants were asked to complete the following: Review model-based specifications by evaluating the accuracy of natural language stakeholder statements represented in the given models. Rate confidence by self-rate confidence in their answers using a five-point scale. Self-rate experience by assessing their experience post-task via a questionnaire on a five-point scale. The study investigated the following: Relationships between self-rated experience, confidence, and performance. Differences between undergraduate and graduate participants.	Comparison Groups: Undergraduates vs. graduates: To assess differences in self-perception, confidence, and performance. Correct vs. incorrect answers: To evaluate confidence ratings. All participants underwent identical experimental tasks tailored to their academic level.	Correctness: Whether a task was completed correctly. Performance: The ratio of correct answers to total tasks. Confidence: Self-rated confidence on task-level answers. Self-rated experience: Average score based on post-task questionnaires. Graduate vs. Undergraduate Comparisons: Performance. Confidence in correct and incorrect answers. Accuracy of self-rated experience relative to performance. Key Findings Confidence: A good predictor of task correctness, regardless of academic level. Self-rated Experience: Not correlated with performance, making it an unreliable predictor. Graduate vs. Undergraduate: Graduates performed better and rated their experience higher. No significant difference in confidence accuracy between the groups.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
28	[59]	Mixed-methods observational study. Purpose: Evaluate the suitability and impact of ChatGPT on students' learning during a five-week introductory Java programming course. Research Questions (RQs): Effect of ChatGPT on learning progress. Suitability for implementation tasks and learning programming concepts. Effort required to adapt ChatGPT-generated code to programming exercises. Application scenarios for ChatGPT use. Reasons for not using ChatGPT.	Conducted at a university offering bachelor's programs in information security. The course was part of a formal undergraduate curriculum.	Participants: A total of 18–22 part-time undergraduate students. Demographics: Students enrolled in a bachelor's program in information security. Experience: The students had completed a previous semester's Python programming course. No prior knowledge of Java programming was assumed.	Duration: Five weeks. Course Structure: Five on-campus lectures. Five programming exercises covering the following: Object-Oriented Programming (OOP); Interfaces and Exception Handling; Collections; File I/O and Streams; Lambda Expressions and Multithreading. Exercises were submitted online for grading and feedback. Use of ChatGPT: Voluntary use for exercise preparation. ChatGPT versions: GPT 3.5 (66.6%) and GPT 4.0 (33.3%). Feedback collected through a 12-question anonymous survey after each exercise.	No explicit control group was included. However, some students chose not to use ChatGPT, providing a natural comparison. Non-users cited the following: Desire to develop programming skills independently. Concerns about misleading or insufficient code. Preference for traditional learning methods.	Learning Progress: Effectiveness of ChatGPT in enhancing students' understanding of programming concepts. Rated positively by most students. No statistically significant relationship between exercises and perceived learning progress ($p = 0.2311$). Suitability for Tasks: Implementation Tasks: Mixed reviews; suitability varied by exercise. No significant relationship between exercises and ratings ($p = 0.4928$). Learning Programming Concepts: Predominantly rated suitable or rather suitable. Statistically significant relationship with exercises ($p = 0.0001$). Adaptation Effort: Minimal effort required to adapt ChatGPT-generated code to tasks. No significant correlation between exercises and adaptation effort ($p = 0.3666$). Application Scenarios: Common uses: Acquiring background knowledge (68%). Learning syntax and concepts (56%). Suggesting algorithms (47%). Used least for reviewing own solutions (28%). Reasons for Non-Use: Concerns about proficiency development. Misleading or incorrect outputs. Preference for independent work. Fundamental rejection of AI tools. Overall Suitability: Positively rated across exercises. Statistically significant correlation between exercises and ratings ($p = 0.0002856$).
29	[16]	Type: Controlled experimental study. Objective: To evaluate the productivity effects of GitHub Copilot, an AI-powered pair programmer, on professional software developers.	Location: Conducted remotely; the participants were recruited globally through Upwork. Setting: Tasks were administered via GitHub Classroom.	Participants: A total of 95 professional software developers recruited via Upwork; 35 completed the task. Age: The majority were aged 25–34. Geographic Distribution: Primarily from India and Pakistan. Education: Predominantly college-educated (4-year degree and above).	Tool: GitHub Copilot, an AI pair programmer powered by OpenAI Codex. Task: Participants were asked to implement an HTTP server in JavaScript as quickly as possible. Process: The treated group was provided with GitHub Copilot and a brief 1 min instructional video on its use.	Comparison: The treated group used GitHub Copilot. The control group relied on traditional methods, including internet resources and their own skills. Task Structure: Both groups were tasked with completing the same programming task under identical conditions (other than access to Copilot).	Task Completion Time: The treated group completed tasks 55.8% faster on average (71.17 min vs. 160.89 min). The improvement was statistically significant ($p = 0.0017$). Task Success: The success rate was 7 percentage points higher in the treated group, though this was not statistically significant. Heterogeneous Effects: Developers with less coding experience, those aged 25–44, and those coding longer daily hours benefited the most. Self-Reported Productivity Gains: The treated and control groups estimated an average productivity gain of 35%, underestimating the measured gain of 55.8%.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
				Coding Experience: An average of 6 years. Workload: An average of 9 h of coding per day. Income: Median annual income between USD 10,000 and USD 19,000.	The group was given installation instructions for GitHub Copilot and were free to use any additional resources, such as internet search and Stack Overflow. The control group did not have access to GitHub Copilot. The group was free to use any external resources, such as internet search and Stack Overflow. Task Administration: A template repository with a skeleton codebase and a test suite was provided. Performance metrics were tracked using timestamps from GitHub Classroom.		Willingness to Pay: The treated group reported a higher average monthly willingness to pay for Copilot (USD 27.25 vs. USD 16.91). Economic Implications: Potential for AI tools, such as Copilot, to broaden access to software development careers by supporting less experienced developers.
30	[42]	Randomised controlled experiment. Objective: To evaluate the effectiveness of student–AI collaborative feedback (hint-writing) on students' learning outcomes in an online graduate-level data science course. Conditions: Baseline: Students independently write hints. AI Assistance: Students write hints with on-demand access to GPT-4-generated hints. AI Revision: Students write hints independently, review GPT-4-generated hints, and revise their hints.	University of Michigan. Course: Online Masters of Applied Data Science program, Data Manipulation course.	Adult learners with introductory knowledge of Python programming and statistics. Total Participants: A total of 97 students took the pre-test; 62 completed both the pre- and post-tests. The students were randomly assigned to the following groups: Baseline (20 students). AI-Assistance (20 students, after propensity score matching). AI-Revision (15 students). Demographics: Graduate students with varying levels of programming proficiency.	Task: Students compared a correct solution to an incorrect solution for a programming assignment and write hints to guide correction of errors. Baseline: Students wrote hints independently. AI-Assistance: Students could access GPT-4-generated hints at any time while writing. AI-Revision: Students wrote independently first, reviewed GPT-4-generated hints, and revised their hints. Programming Tools: JupyterLab and Python. Assignment Grading: Programming assignments were automatically graded using the Nbgrader tool. Implementation: Assignments included an example task with guidance on writing effective hints. Incorrect solutions were selected using a similarity-based metric from a repository of prior incorrect submissions.	Group: Baseline condition. Task: Students wrote hints independently, with no AI support.	Learning Outcomes: Pre-test: Assessed Python programming knowledge (10 MCQs, non-graded). Post-test: Assessed debugging and data manipulation skills (six MCQs, graded, worth 5% of the course grade). Findings: AI-Revision showed higher post-test scores than AI-Assistance and Baseline, though not statistically significant ($p = 0.18$). AI-Assistance showed the lowest mean scores, indicating potential over-reliance on AI hints. Student Engagement: Positive feedback on hint-writing assignments, especially in the AI-Revision condition. Students valued the activity for improving debugging and critical thinking skills. Behavioural Insights: AI-Revision promoted critical evaluation and refinement of hints, enhancing learning. AI-Assistance encouraged reliance on AI-generated content, reducing independent effort.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
31	[60]	Type: Crossover experimental study. Objective: To evaluate the effectiveness of just-in-time teaching interventions in improving the pedagogical practices of teaching assistants (TAs) during online one-on-one programming tutoring sessions. Intervention Duration: Participants received interventions immediately before each tutoring session, with each session lasting approximately one hour. Key Variables: Independent Variable: Presence or absence of the just-in-time teaching intervention. Dependent Variables: Duration and proportion of productive teaching events, tutor talk time, and self-reported perceptions.	Location: Conducted at a university computer science department. Environment: Online setting using a simulated tutoring scenario.	Participants: A total of 46 university students. Composition: Graduate and undergraduate computer science students. Recruitment: Recruited from department mailing lists. Demographics: Mix of experienced and novice tutors, with diverse teaching interests and abilities.	Treatment Group: Shown a "teaching tips" screen before the tutoring session. Included pedagogical advice, including the following: Asking open-ended questions. Checking for student understanding. Encouraging the student to talk more during the session. Information about each student's lecture attendance. Control Group: Shown a logistical tips screen focusing on meeting setup and technical instructions (e.g., camera and microphone settings). Tutoring Task: Participants roleplayed as tutors for an introductory programming task (FizzBuzz). Sessions featured a researcher acting as a student with two versions of buggy code to ensure variety	Received no pedagogical advice. Exposed only to logistical reminders and technical tips before the tutoring session.	Primary Outcomes: Productive Teaching Events: Time spent engaging students with effective teaching techniques. Proportion of session duration devoted to productive interactions. Tutor Talk Time: Ratio of tutor to student speaking time. Secondary Outcomes: Participants' ability to transfer learned teaching behaviours to subsequent sessions. Perceived usefulness of the intervention from participant interviews. Key Findings: Participants in the treatment condition spent significantly more time in productive teaching events (1.4 times increase, Cohen's d = 0.72). Treatment significantly reduced tutor talk time, increasing opportunities for student participation ($p < 0.05$). Evidence of behaviour transfer to subsequent sessions was inconclusive but self-reported by 16 of 22 treatment-first participants.
32	[61]	Controlled quasi-experiment Duration: Ten weeks. Purpose: Investigate the impacts of different designs of automated formative feedback on student performance, interaction with the feedback system, and perception of the feedback.	Conducted at a large university in the Pacific Northwest of the United States.	Participants: A total of 76 students enrolled in a CS2 course. Group Assignment: Students were randomly assigned to three different lab sections, each treated as a group. Characteristics: The study included diverse participants in terms of programming experience and demographics (not explicitly detailed in the study).	Feedback Types: Knowledge of Results (KR): Information on whether a test case passed or failed. Knowledge of Correct Responses (KCR): KR + detailed comparisons between expected and actual outputs. Elaborated Feedback (EF): KCR + one-level hints addressing common mistakes with additional explanations for misconceptions. Feedback Delivery: Automated feedback delivered through a system integrating GitHub, Gradle, and Travis-CI. Group Assignments: Group KR: Received KR feedback (25 students). Group KCR: Received KR + KCR feedback (25 students). Group EF: Received KR + KCR + EF feedback (26 students).	Baseline: Group KR served as the control group, receiving the least detailed feedback. Absence of a no-feedback group: Deliberately excluded as research shows providing no feedback is less effective.	Student Performance: Measured by the percentage of passed test cases across three programming assignments. Student Interaction with the Feedback System: Metrics: Number of feedback requests (pushes to GitHub). Efforts evidenced by the number of changed lines of code. Behavioural Patterns: How students interacted with and utilised feedback. Student Perceptions: Assessed using a four-point Likert scale survey and open-ended questions addressing the following: Frequency of feedback use. Experience in interpreting and utilising feedback. Likes/dislikes about the feedback system. Suggestions for improvement.

Table A1. *Cont.*

N	Authors	Study Design	Study Region	Population Characteristics	Intervention Details	Control Group Details	Outcomes Measured
33	[23]	Pre-test–post-test quasi-experimental design.	Two universities in North Cyprus.	A total of 50 undergraduate students.	Experimental Group: Used ChatGPT for solving quizzes.	Performed the quizzes without ChatGPT assistance.	Comparison of AI-assisted vs. manual performance.
34	[62]	Quasi-experimental design with two groups (experimental and control).	Saudi Arabia.	Tenth-grade female students (N = 37), randomly assigned into experimental (N = 19) and control (N = 18) groups.	The experimental group was taught HTML programming using gamification elements (points, leaderboards, badges, levels, progress bars, rewards, avatars).	The control group was taught HTML programming using the traditional teaching method.	Programming skills (HTML tags, paragraphs, lists, multimedia, hyperlinks) and academic achievement motivation (desire to excel, goal orientation, academic persistence, academic competition, academic achievement behaviour, enjoyment of programming).
35	[17]	Quasi-experimental study comparing two groups (control vs. experimental) using a programming challenge.	Conducted in a university setting (Prince Sultan University, College of Computer and Information Sciences).	Twenty-four undergraduate students (CS majors) who had completed CS101, CS102, and CS210 with a minimum grade of C+.	Group A (control group): Used textbooks and notes without internet access. Group B (experimental group): Had ChatGPT access for solving programming challenges.	The control group (Group A) provided a realistic benchmark for evaluating the impact of ChatGPT. The group's reliance on traditional learning resources led to slower but potentially more structured problem-solving approaches. However, the group's lower overall scores and longer debugging times indicated that AI-assisted learning (Group B) had a clear advantage in speed and performance, albeit with accuracy challenges.	1. Programming performance (scores)—Number of passed test cases. 2. Time taken—Efficiency in solving problems. 3. Code accuracy and debugging effort—Issues due to ChatGPT-generated code.

References

1. Arango, M.C.; Hincapie-Otero, M.; Hardeman, K.; Shao, B.; Starbird, L.; Starbird, C. Special considerations for the use of AI tools by PEERs as a learning and communication aid. *J. Cell. Physiol.* **2024**, *239*, e31339. [CrossRef] [PubMed]
2. Arun, G.; Perumal, V.; Urias, F.; Ler, Y.E.; Tan, B.W.T.; Vallabhajosyula, R.; Tan, E.; Ng, O.; Ng, K.B.; Mogali, S.R. ChatGPT versus a customized AI chatbot (Anatbuddy) for anatomy education: A comparative pilot study. *Anat. Sci. Educ.* **2024**, *17*, 1396–1405. [CrossRef]
3. Carvalho, W.; Tomov, M.S.; de Cothi, W.; Barry, C.; Gershman, S.J. Predictive Representations: Building Blocks of Intelligence. *Neural Comput.* **2024**, *36*, 2225–2298. [CrossRef] [PubMed]
4. Chen, D.; Liu, W.; Liu, X. What drives college students to use AI for L2 learning? Modeling the roles of self-efficacy, anxiety, and attitude based on an extended technology acceptance model. *Acta Psychol.* **2024**, *249*, 104442. [CrossRef] [PubMed]
5. Chiappa, A.S.; Tano, P.; Patel, N.; Ingster, A.; Pouget, A.; Mathis, A. Acquiring musculoskeletal skills with curriculum-based reinforcement learning. *Neuron* **2024**, *112*, 3969–3983.e5. [CrossRef]
6. Choudhury, A.; Chaudhry, Z. Large Language Models and User Trust: Consequence of Self-Referential Learning Loop and the Deskilling of Health Care Professionals. *J. Med. Internet. Res.* **2024**, *26*, e56764. [CrossRef]
7. Grandchamp des Raux, H.; Ghilardi, T.; Soderberg, C.; Ossmy, O. The role of action concepts in physical reasoning: Insights from late childhood. *Philos. Trans. R Soc. Lond. B Biol. Sci.* **2024**, *379*, 20230154. [CrossRef]
8. Hultberg, P.T.; Santandreu Calonge, D.; Kamalov, F.; Smail, L. Comparing and assessing four AI chatbots' competence in economics. *PLoS ONE* **2024**, *19*, e0297804. [CrossRef]
9. Hussain, K.; Nso, N.; Tsourdinis, G.; Haider, S.; Mian, R.; Sanagala, T.; Erwin, J.P., 3rd; Pursnani, A. A systematic review and meta-analysis of left atrial strain in hypertrophic cardiomyopathy and its prognostic utility. *Curr. Probl. Cardiol.* **2024**, *49 (1 Pt C)*, 102146. [CrossRef]
10. Illouz, T.; Ascher, L.A.B.; Madar, R.; Okun, E. Unbiased analysis of spatial learning strategies in a modified Barnes maze using convolutional neural networks. *Sci. Rep.* **2024**, *14*, 15944. [CrossRef]
11. Jallad, S.T.; Alsaqer, K.; Albadareen, B.I.; Al-Maghaireh, D. Artificial intelligence tools utilized in nursing education: Incidence and associated factors. *Nurse Educ. Today* **2024**, *142*, 106355. [CrossRef] [PubMed]
12. Kitchenham, B.; Brereton, O.P.; Budgen, D.; Turner, M.; Bailey, J.; Linkman, S. Systematic literature reviews in software engineering—A systematic literature review. *Inf. Softw. Technol.* **2009**, *51*, 7–15. [CrossRef]
13. Maher, M.L.; Tadimalla, S.Y.; Dhamani, D. An Exploratory Study on the Impact of AI tools on the Student Experience in Programming Courses: An Intersectional Analysis Approach. In 2023 IEEE Frontiers in Education Conference (FIE), IEEE, College Station, TX, USA, 18–21 October 2023; pp. 1–5.
14. Sinha, A.; Goyal, S.; Sy, Z.; Kuperus, R.; Dickey, E.; Bejarano, A. BoilerTAI: A platform for enhancing instruction using generative AI in educational forums. *arXiv* **2024**, arXiv:2409.13196.
15. De Silva, D.I.; Vidhanaarachchi, S.; Kariyawasam, S.B.; Dasanayake, L.R.S.; Thawalampola, O.D.; Jayasuriya, T.D.D.H. CodeCoach: An interactive programming assistance tool. *J. Propuls. Technol.* **2023**, *44*, 7281–7288.
16. Peng, S.; Kalliamvakou, E.; Cihon, P.; Demirer, M. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv* **2023**, arXiv:2302.06590.
17. Qureshi, B. Exploring the use of chatgpt as a tool for learning and assessment in undergraduate computer science curriculum: Opportunities and challenges. *arXiv* **2023**, arXiv:2304.11214.
18. Wang, W.; Ning, H.; Zhang, G.; Liu, L.; Wang, Y. Rocks coding, not development: A human-centric, experimental evaluation of LLM-supported SE tasks. *Proc. ACM Softw. Eng.* **2024**, *1*, 699–721. [CrossRef]
19. Xue, Y.; Chen, H.; Bai, G.R.; Tairas, R.; Huang, Y. Does ChatGPT help with introductory programming? An experiment of students using ChatGPT in CS1. In Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, Lisbon, Portugal, 14–20 April 2024; pp. 331–341.
20. Kosar, T.; Ostojić, D.; Liu, Y.D.; Mernik, M. Computer science education in the ChatGPT era: Experiences from an experiment in a programming course for novice programmers. *Mathematics* **2024**, *12*, 629. [CrossRef]
21. Leinonen, J.; Denny, P.; MacNeil, S.; Sarsa, S.; Bernstein, S.; Kim, J.; Tran, A.; Hellas, A. Comparing code explanations created by students and large language models. In Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1, Turku, Finland, 7–12 June 2023; pp. 124–130.
22. Ouyang, F.; Guo, M.; Zhang, N.; Bai, X.; Jiao, P. Comparing the effects of instructor manual feedback and ChatGPT intelligent feedback on collaborative programming in China's higher education. *IEEE Trans. Learn. Technol.* **2024**, *17*, 2227–2239. [CrossRef]
23. Al Hajj, J.; Sah, M. Assessing the impact of ChatGPT in a PHP programming course. In Proceedings of the 2023 7th International Symposium on Innovative Approaches in Smart Technologies (ISAS), Istanbul, Turkiye, 23–25 November 2023; IEEE: Piscataway, NJ, USA; pp. 1–10.

24. Alshaikh, Z.; Tamang, L.J.; Rus, V. Experiments with a Socratic intelligent tutoring system for source code understanding. In Proceedings of the Thirty-Third International Florida Artificial Intelligence Research Society Conference (FLAIRS-32), North Miami Beach, FL, USA, 17–20 April 2020.
25. Jarry Trujillo, C.; Vela Ulloa, J.; Escalona Vivas, G.; Grasset Escobar, E.; Villagran Gutierrez, I.; Achurra Tirado, P.; Varas Cohen, J. Surgeons vs ChatGPT: Assessment and Feedback Performance Based on Real Surgical Scenarios. *J. Surg. Educ.* **2024**, *81*, 960–966. [CrossRef]
26. Khan, K.; Katarya, R. WS-BiTM: Integrating White Shark Optimization with Bi-LSTM for enhanced autism spectrum disorder diagnosis. *J. Neurosci. Methods* **2025**, *413*, 110319. [CrossRef] [PubMed]
27. Koch, E.T.; Cheng, J.; Ramandi, D.; Sepers, M.D.; Hsu, A.; Fong, T.; Murphy, T.H.; Yttri, E.; Raymond, L.A. Deep behavioural phenotyping of the Q175 Huntington disease mouse model: Effects of age, sex, and weight. *BMC Biol.* **2024**, *22*, 121. [CrossRef] [PubMed]
28. Zha, F.; Wang, Y.; Mao, L.; Liu, J.; Wang, X. Can university marks measure programming skills for novice programmers? An exploratory study. *J. Internet Technol.* **2023**, *24*, 1189–1197.
29. Lawson McLean, A. Constructing knowledge: The role of AI in medical learning. *J. Am. Med. Inform. Assoc.* **2024**, *31*, 1797–1798. [CrossRef]
30. Li, W.; Zhang, X.; Li, J.; Yang, X.; Li, D.; Liu, Y. An explanatory study of factors influencing engagement in AI education at the K-12 Level: An extension of the classic TAM model. *Sci. Rep.* **2024**, *14*, 13922. [CrossRef]
31. Macnamara, B.N.; Berber, I.; Cavusoglu, M.C.; Krupinski, E.A.; Nallapareddy, N.; Nelson, N.E.; Smith, P.J.; Wilson-Delfosse, A.L.; Ray, S. Does using artificial intelligence assistance accelerate skill decay and hinder skill development without performers' awareness? *Cogn. Res. Princ. Implic.* **2024**, *9*, 46. [CrossRef]
32. Maldonado-Trapp, A.; Bruna, C. The Evolution of Active Learning in Response to the Pandemic: The Role of Technology. *Adv. Exp. Med. Biol.* **2024**, *1458*, 247–261. [CrossRef]
33. Marchesi, S.; De Tommaso, D.; Kompatsiari, K.; Wu, Y.; Wykowska, A. Tools and methods to study and replicate experiments addressing human social cognition in interactive scenarios. *Behav. Res. Methods* **2024**, *56*, 7543–7560. [CrossRef]
34. Moulin, T.C. Learning with AI Language Models: Guidelines for the Development and Scoring of Medical Questions for Higher Education. *J. Med. Syst.* **2024**, *48*, 45. [CrossRef]
35. Naamati-Schneider, L. Enhancing AI competence in health management: Students' experiences with ChatGPT as a learning Tool. *BMC Med. Educ.* **2024**, *24*, 598. [CrossRef]
36. Pinheiro, E.D.; Sato, J.R.; Junior, R.; Barreto, C.; Oku, A.Y.A. Eye-tracker and fNIRS: Using neuroscientific tools to assess the learning experience during children's educational robotics activities. *Trends Neurosci. Educ.* **2024**, *36*, 100234. [CrossRef] [PubMed]
37. Singaram, V.S.; Pillay, R.; Mbobnda Kapche, E.L. Exploring the role of digital technology for feedback exchange in clinical training: A scoping review. *Syst. Rev.* **2024**, *13*, 298. [CrossRef] [PubMed]
38. Sun, G.H. Prompt Engineering for Nurse Educators. *Nurse Educ.* **2024**, *49*, 293–299. [CrossRef] [PubMed]
39. Manley, E.D.; Urness, T.; Migunov, A.; Reza, M.A. Examining student use of AI in CS1 and CS2. *J. Comput. Sci. Coll.* **2024**, *39*, 41–51.
40. Wang, Z.; Wang, S.; Wang, M.; Sun, Y. Design of application-oriented disease diagnosis model using a meta-heuristic algorithm. *Technol. Health Care* **2024**, *32*, 4041–4061. [CrossRef]
41. Choudhuri, R.; Liu, D.; Steinmacher, I.; Gerosa, M.; Sarma, A. How far are we? The triumphs and trials of generative AI in learning software engineering. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon, Portugal, 20 April 2024; pp. 1–13.
42. Singh, A.; Brooks, C.; Wang, X. The impact of student-AI collaborative feedback generation on learning outcomes. In Proceedings of the AI for Education: Bridging Innovation and Responsibility at the 38th AAAI Annual Conference on AI, Vancouver, BC, Canada, 20–27 February 2024.
43. Nguyen, S.; Babe, H.M.; Zi, Y.; Guha, A.; Anderson, C.J.; Feldman, M.Q. How beginning programmers and code LLMs (mis) read each other. In Proceedings of the CHI Conference on Human Factors in Computing Systems, Honolulu, HI, USA, 11–16 May 2024; pp. 1–26.
44. Ouaazki, A.; Bergram, K.; Farah, J.C.; Gillet, D.; Holzer, A. Generative AI-enabled conversational interaction to support self-directed learning experiences in transversal computational thinking. In Proceedings of the 6th ACM Conference on Conversational User Interfaces, Luxembourg, Luxembourg, 8–10 July 2024; pp. 1–12.
45. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [CrossRef]
46. Seferoglu, S.S. A comparison of solo and pair programming in terms of flow experience, coding quality, and coding achievement. *J. Educ. Comput. Res.* **2021**, *58*, 1448–1466.

47. Manfredi, G.; Erra, U.; Gilio, G. A mixed reality approach for innovative pair programming education with a conversational AI virtual avatar. In Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering, Oulu, Finland, 14–16 June 2023; pp. 450–454.
48. Liu, D.; Calver, J.; Craig, M. A Static Analysis Tool in CS1: Student Usage and Perceptions of PythonTA. In Proceedings of the 26th Australasian Computing Education Conference, Sydney, NSW, Australia, 29 January–2 February 2024; pp. 172–181.
49. Coffman, J.; de Freitas, A.A.; Hill, J.M.; Weingart, T. Visual vs. textual programming languages in CS0.5: Comparing student learning with and student perception of RAPTOR and Python. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1, Toronto, ON, Canada, 15–18 March 2023; pp. 32–38.
50. Obermüller, F.; Greifenstein, L.; Fraser, G. Effects of automated feedback in scratch programming tutorials. In Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1, Turku, Finland, 7–12 June 2023; pp. 396–402.
51. Kristiansen, N.G.; Nicolajsen, S.M.; Brabrand, C. Feedback on student programming assignments: Teaching assistants vs. automated assessment tool. In Proceedings of the 23rd Koli Calling International Conference on Computing Education Research, Koli, Finland, 13–18 November 2023; 10p.
52. Scholl, A.; Kiesler, N. How novice programmers use and experience ChatGPT when solving programming exercises in an introductory course. *arXiv* **2024**, arXiv:2407.20792.
53. Kazemitabaar, M.; Hou, X.; Henley, A.; Ericson, B.J.; Weintrop, D.; Grossman, T. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. In Proceedings of the 23rd Koli Calling International Conference on Computing Education Research, Koli, Finland, 13–18 November 2023; 12p.
54. Ma, Q.; Shen, H.; Koedinger, K.; Wu, T. How to teach programming in the AI era? Using LLMs as a teachable agent for debugging. *arXiv* **2023**, arXiv:2310.05292.
55. Fan, Z.; Noller, Y.; Dandekar, A.; Roychoudhury, A. Intelligent tutoring system: Experience of linking software engineering and programming teaching. *arXiv* **2023**, arXiv:2310.05472.
56. Pankiewicz, M.; Baker, R.S. Large language models (GPT) for automating feedback on programming assignments. *arXiv* **2023**, arXiv:2307.00150. [CrossRef]
57. Wang, X.; Wang, Y.; Yang, F.; Le, W.; Wang, S. Measuring programming ability for novice programmers. *J. Internet Technol.* **2022**, *23*, 573–581. [CrossRef]
58. Daun, M.; Brings, J.; Obe, P.A.; Stenkova, V. Reliability of self-rated experience and confidence as predictors for students' performance in software engineering: Results from multiple controlled experiments on model comprehension with graduate and undergraduate students. *Empir. Softw. Eng.* **2021**, *26*, 80. [CrossRef]
59. Haindl, P.; Weinberger, G. Students' experiences of using ChatGPT in an undergraduate programming course. *IEEE Access* **2024**, *12*, 43519–43529. [CrossRef]
60. Cheng, A.Y.; Tanimura, E.; Tey, J.; Wu, A.C.; Brunskill, E. Brief, just-in-time teaching tips to support computer science tutors. In Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1, Portland, OR, USA, 20–23 March 2024; pp. 200–206.
61. Hao, Q.; Smith IV, D.H.; Ding, L.; Ko, A.; Ottaway, C.; Wilson, J.; Greer, T. Towards understanding the effective design of automated formative feedback for programming assignments. *Comput. Sci. Educ.* **2022**, *32*, 105–127. [CrossRef]
62. Alsuhaymi, D.S.; Alotaibi, O.M. Gamification's Efficacy in Enhancing Students' HTML Programming Skills and Academic Achievement Motivation. *J. Educ. E-Learn. Res.* **2023**, *10*, 397–407. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.